∞ MMLA-RTT: Dual-State Learning at Reasoning Time

MMLA Focus Paper

Junyi Zou[1,*] • **Avrova Donz**[1,2,*]

[1]MMLA-org [2]Communication University of China (CUC), No. 1 Dingfuzhuang East Street, Chaoyang District, Beijing 100024, China

[*]Equal contribution

Email: zoujunyi@zjydiary.cn · avrovadonz@icloud.com

Project: github.com/MMLA-org/mmla-memory

ABSTRACT

Reasoning-time systems can change in two fundamentally different ways while one problem remains unresolved. A trainable numerical policy state Φ can absorb within-problem feedback and alter later attempts. A bounded authoritative memory M can commit typed, versioned, provenance-bearing rows for later retrieval. We formalize their composition without treating them as one hidden state. The two carriers have different types, writers, readers, privileges, lifetimes, reset operations, rollback domains, ledgers, and failure modes. An attempt pins a read-consistent view tuple (v^{Φ}, v^{M}), but tuple pinning neither creates joint ownership nor permits one carrier's reset to restore the other.

We define a fine-grained interleaving schedule over generation, feedback, policy update, candidate construction, trusted memory admission, and later reuse. Strict reasoning-time learning requires that feedback from an earlier attempt update Φ and that a later attempt on the same unresolved problem causally read the updated policy state. Memory mutation alone is reasoning-time memory adaptation, not policy learning. Under explicit measurability, bounded-update, authorization, compatibility, and intervention assumptions, we prove type non-conflation, strict-RTT classification, independent rollback preservation, conditional commutation, bounded policy drift, a contamination bound for indiscriminate compression, a matching counterexample where compression is sufficient, interaction-sensitive error bounds, and identification of a 2×2 policy-by-memory factorial contrast. We also prove that output traces alone cannot identify which carrier changed.

The paper supplies reset, swap, freeze, rollback, and receipt controls; complete training and deployment cost ledgers; failure and recovery tables; a staged factorial protocol; and theorem-to-implementation obligations. It reports no new experiment and no empirical dual-state success. The same-checkpoint semantic interface remains failed/unqualified, so authoritative memory composition, an oracle gap, predictive admission, and any learned admission policy remain unvalidated or unopened. All results are conditional mathematics or planned falsification tests.

1 Introduction

1.1 The composition problem

A model may improve a later attempt because its numerical behavior changed, because an authoritative record changed, because more tokens were placed in context, or because sampling happened to differ. Those mechanisms are not interchangeable. This paper studies the first two as a typed composition. It asks what must be logged, intervened on, and proved before a within-problem policy update and an authoritative memory commit can be said to coexist without hidden coupling.

DEFINITION

Definition 1.1 (Dual-state reasoning-time system)**.** A dual-state reasoning-time system has a trainable numerical policy carrier $\Phi \in \mathcal{P}$ and an authoritative memory carrier $M \in \mathcal{M}$. Its attempt kernel reads a declared causal workspace together with pinned versions of both carriers. The carriers expose disjoint mutation interfaces, version lines, rollback domains, and ledgers. A coordinator may check a cross-carrier compatibility predicate but does not own either state.

The definition does not assert that such a system has been implemented, trained, or improved. It rules out a common shortcut: calling one opaque recurrent vector "policy and memory" and then inferring separate causal effects from a single output trace.

1.2 Scientific boundary

The public MMLA V3 manuscript and its complete technical-report evidence record provide context [12, 13]. Their current blocker is unchanged. PM-I2 qualified 0/9 registered same-checkpoint interfaces. In the fitted-population diagnostic, the direct candidate was exact on 40/73,728 probes; learned-route/learned-content and oracle-route/learned-content were each 0/73,728; learned-route/oracle-content was 18,544/73,728, exactly the routing count; and only the complete mechanical oracle reached 73,728/73,728. Preservation was zero and 43,008 cases were missing. The result is fit/support failure for the tested pure latent-row interface. It does not isolate representation, decoder, or optimization and does not reject every MMLA realization.

UNVALIDATED Therefore this paper does not claim a qualified semantic row interface, empirical authoritative-memory success, an expected-risk oracle gap, predictive admission, a learned value model, or a learned admission policy. The proposed composition cannot be an empirical success while its authoritative-memory prerequisite remains unqualified.

Three companion focus papers contribute vocabulary and formal contracts only. RTT Foundations distinguishes policy state from memory state [14]. Atomic Memory Rows defines complete authoritative rows and exact NULL [10]. Predictive Memory Admission defines a future-risk admission problem but reports no oracle gap or learned selector [11]. No arrow in Figure 1 transfers empirical status.

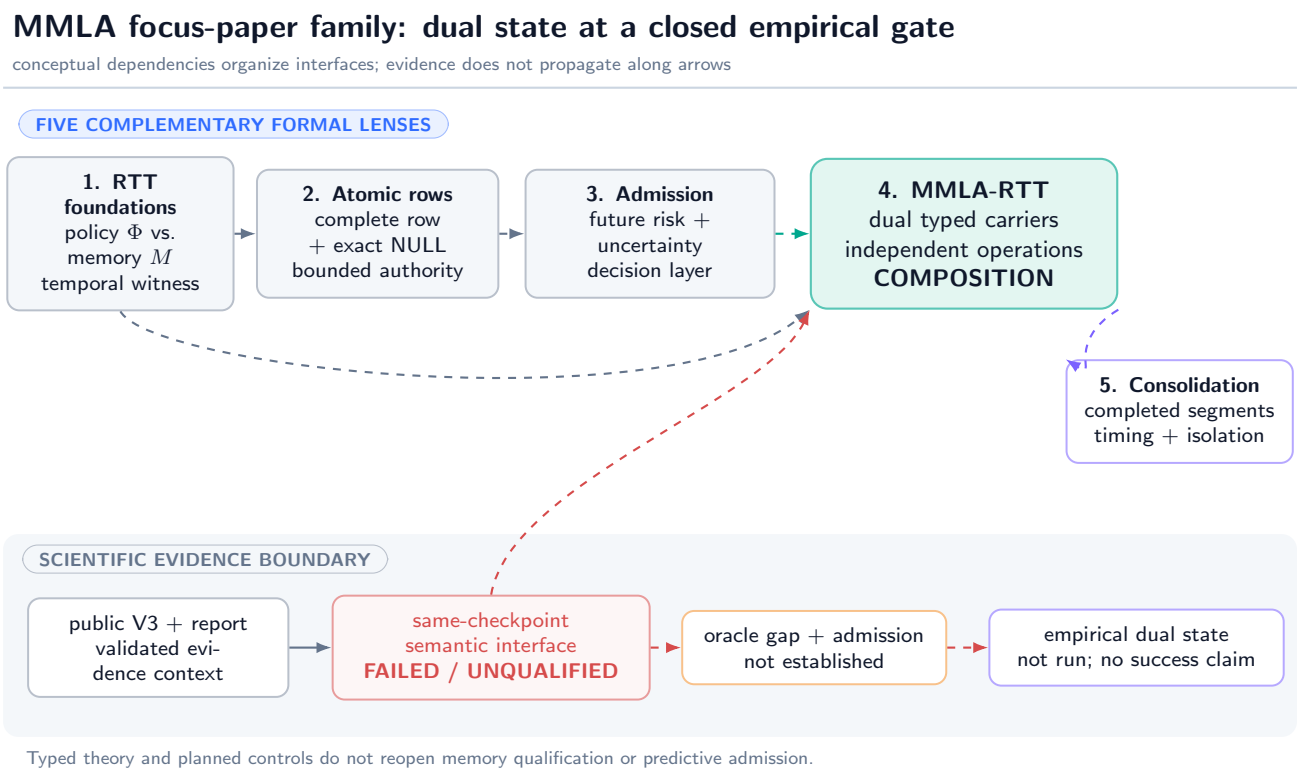

Figure 1: **Dual-state composition in the five-paper conceptual family.** The semantic-interface stop is inherited and remains closed; formal prerequisites do not provide empirical validation.

1.3 Contributions and nonclaims

The paper develops typed semantics, an explicit causal schedule, independent operation algebras, mathematical bounds, factorial estimands, negative controls, resource ledgers, and falsification obligations. It does not supply a semantic serializer, a trusted row assembler implementation, an empirical policy update, a memory-admission model, a dual-state checkpoint, or a deployment result. Definitions, conditional proofs, prescriptions, and planned tests are marked locally and cannot promote an implementation claim.

1.4 Central falsifiable questions

Let Y_{pm} be a preregistered loss under policy intervention $p \in \{0, 1\}$ and memory intervention $m \in \{0, 1\}$. The primary interaction estimand is

$$\Delta_{\Phi M} = \mathbb{E}[Y_{11} - Y_{10} - Y_{01} + Y_{00}], \tag{1}$$

where lower loss is better. A negative value would indicate complementarity on that endpoint under the declared interventions; positive values indicate antagonism. The paper proves how this contrast is identified under randomization and compliance, not

what its sign is. A second question is categorical: does feedback at attempt r update Φ and causally influence attempt $r+1$ on the same still-unresolved problem? Without that path, the mechanism is not strict reasoning-time learning even if M changes.

CONJECTURED Independent policy adaptation and authoritative memory may be complementary in some domains, substitutable in others, neutral when one carrier is irrelevant, and harmful when either update contaminates the other carrier's inputs. All four regimes remain possible before intervention.

2 Typed Carriers and Read Views

2.1 Probability space and causal workspace

Fix a probability space $(\Omega, \mathcal{F}, \mathbb{P})$ and a problem instance q. Attempts are indexed by $r = 0, 1, \ldots, R_q$. Let $\mathcal{F}_{q,r,t}$ be the information available immediately before micro-event t of attempt r. A bounded causal workspace

$$W_{q,r,t} = \big(q,\ x_{q,r,\le t},\ z_{q,<r},\ f_{q,<r},\ u_{q,r,t}\big) \tag{2}$$

contains the problem, visible prefix, prior public attempt summaries z, admissible feedback f, and a declared controller summary u. Raw private chain-of-thought is neither an authoritative record nor a necessary audit artifact. Any feature used to update either carrier must be measurable with respect to its declared information set.

DEFINITION

Definition 2.1 (Policy carrier)**.** The policy carrier at event e is

$$P_e = (\Phi_e, O_e, v_e^{\Phi}, \ell_e^{\Phi}, \tau_e^{\Phi}, \rho_e^{\Phi}), \tag{3}$$

where $\Phi_e \in \mathcal{P} \subseteq \mathbb{R}^d$ is trainable numerical state, O_e is optimizer state, v_e^{Φ} is a monotone version, ℓ_e^{Φ} is a lineage identifier, τ_e^{Φ} declares lifetime and scope, and ρ_e^{Φ} is a receipt pointer. Its privileged writer is the policy updater; its readers are declared inference modules.

DEFINITION

Definition 2.2 (Authoritative memory carrier)**.** The memory carrier at event e is

$$A_e = (M_e, v_e^{M}, \ell_e^{M}, \tau_e^{M}, \rho_e^{M}), \tag{4}$$

where M_e is a bounded map from physical slots to complete typed rows or empty slots, v_e^{M} is an authoritative memory version, ℓ_e^{M} is its lineage, τ_e^{M} declares retention and tenant scope, and ρ_e^{M} points to a commit or NULL receipt. Its privileged writer is a trusted memory assembler/committer; policy gradients never directly mutate it.

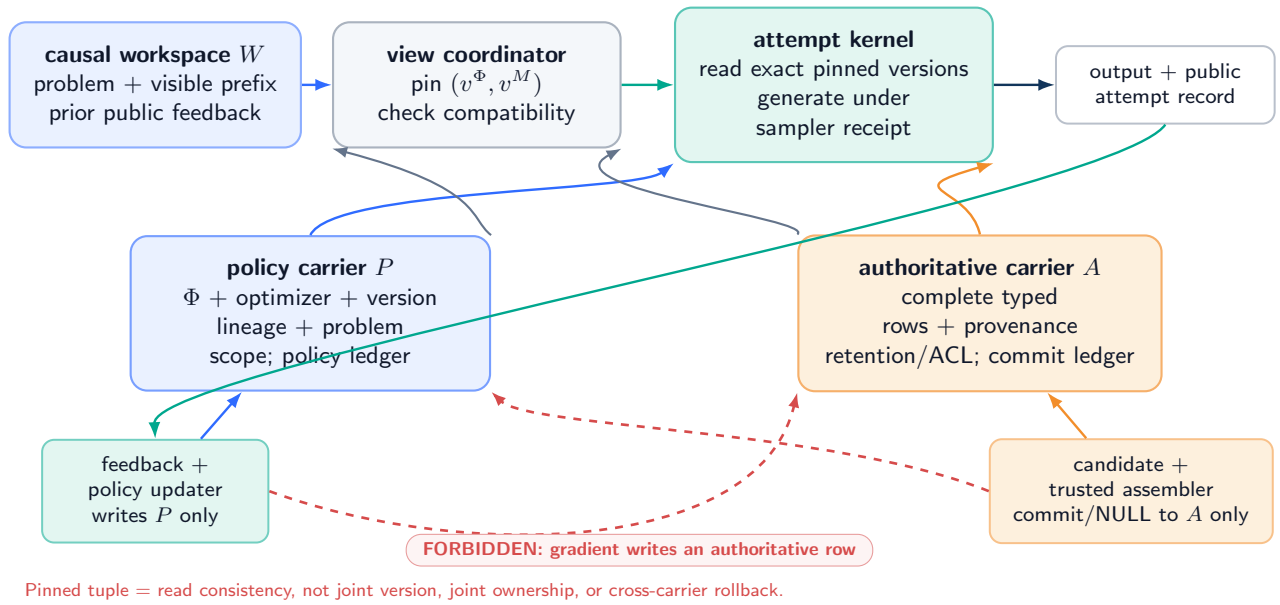

Figure 2: **Typed dual-state architecture** (DEFINITION / UNVALIDATED)**.** The attempt kernel reads a pinned view. Feedback may drive a policy update; a separate trusted path may assemble and commit a complete row. Dashed coral paths are forbidden privilege crossings.

2.2 Type, privilege, lifetime, and failure separation

Property	Policy carrier P	Memory carrier A
Payload	Numerical parameters, optimizer moments, update metadata	Complete typed rows, versions, provenance, ACL/protection, validity, tombstone semantics
Privileged writer	Policy-update service under feedback contract	Trusted assembler and atomic committer
Typical lifetime	Attempt/problem/session; default problem-scoped	Cross-attempt and potentially persistent/archival under retention policy
Reset target	Chosen policy baseline and optimizer state	Chosen authoritative snapshot or empty/default row set
Rollback unit	Numerical checkpoint plus optimizer/lineage receipt	Full row version or full memory snapshot plus commit ledger
Failure examples	Divergence, reward hacking, optimizer corruption, catastrophic local drift	Partial row, stale authority, ACL breach, provenance loss, rollback fork
Primary ledger	Gradient/update, loss/reward, seed, optimizer, version	Candidate, validation, commit/NULL, row version, archive, index, deletion

Table 1: The carriers are distinct across every operational dimension required by this paper.

DEFINITION

Definition 2.3 (Pinned read view)**.** At the start of attempt r, the coordinator emits

$$V_{q,r} = \left(v^{\Phi}_{q,r}, v^{M}_{q,r}, \ell^{\Phi}_{q,r}, \ell^{M}_{q,r}, c_{q,r}\right), \tag{5}$$

where $c_{q,r}$ is a compatibility-receipt identifier. The attempt kernel may read only the named carrier versions. Pinning provides read consistency for that attempt; it does not merge carriers, equate versions, assign joint ownership, or imply joint commit/rollback.

Assumption 2.4 (Typed authorization)**.** Every state-changing event carries a carrier tag, actor identity, scope, predecessor version, proposed successor version, payload hash, and authorization decision. A policy event can write only P; a memory event can write only A. The coordinator can reject an incompatible view but cannot synthesize either successor.

PROVED UNDER STATED ASSUMPTIONS

Proposition 2.5 (Syntactic non-conflation)**.** *Under Assumption 2.4, no well-typed execution trace can apply a policy mutation to M or a memory commit to Φ. Equality of serialized payload bytes, if it occurs, does not change the carrier tag or privilege domain.*

The proposition is a property of the specified interface, not evidence that an implementation enforces it. Its proof is in Appendix A.

2.3 Complete state and carrier inventory

The system state at event e is not just (Φ_e, M_e). For causal and cost completeness we use

$$S_e = (W_e, P_e, A_e, C_e, G_e, Q_e), \tag{6}$$

where C_e is immutable causal context, G_e is generation/sampler state including seed and decoding controls, and Q_e contains queues, snapshots, caches, and external ledgers. Omitting G_e can misattribute sampling variation to learning. Omitting Q_e can hide stale cache reads or delayed commits. Neither auxiliary state becomes authoritative memory merely by appearing in the tuple.

3 Fine-Grained Causal Timeline

3.1 Micro-events, not an opaque recurrence

Let $e = 0, 1, \ldots$ index globally ordered micro-events. Each event has one of the tags

$$\mathsf{GEN},\ \mathsf{OBS},\ \mathsf{FDBK},\ \mathsf{PUPD},\ \mathsf{CAND},\ \mathsf{MVAL},\ \mathsf{MCOM},\ \mathsf{PIN},\ \mathsf{RESET},\ \mathsf{ROLLBACK}. \tag{7}$$

The transition is

$$S_{e+1} = T_{\kappa_e}(S_e, \xi_e), \tag{8}$$

where κ_e is the event tag and ξ_e is its typed input. There is no unlabelled map $S_{e+1} = F(S_e)$ from which causal ownership must be guessed after the fact.

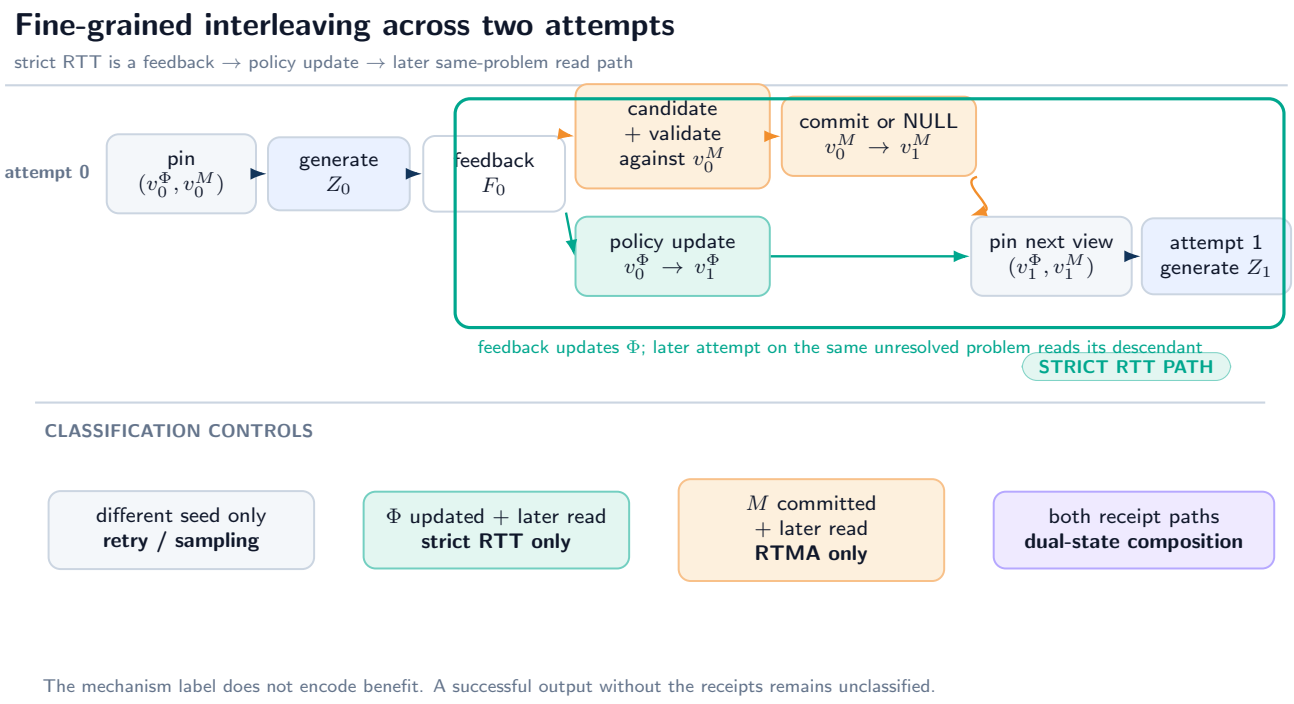

Figure 3: **Declared interleaving for two attempts (DEFINITION).** Attempt 0 pins a view, generates, receives admissible feedback, and may trigger two independent successor events. Attempt 1 counts as strict RTT only if it reads the updated Φ while the problem remains unresolved. A memory commit has its own later-reuse path.

For one illustrative order, attempt r pins (v_r^Φ, v_r^M), generates Z_r, obtains feedback F_r, performs a policy update to v_{r+1}^Φ, independently proposes and validates a memory candidate, optionally commits v_{r+1}^M, and then pins a new view for attempt

$r + 1$. Other orders are permitted only when declared. In particular, memory admission may precede policy update, or either event may be absent. Section 4 shows why the order can matter.

3.2 Filtration and no future leakage

Assumption 3.1 (Causal measurability)**.** At event e, generation and policy-update inputs are $\mathcal{F}_e$-measurable. Memory candidates may use only the candidate-construction information set declared for that event. A deployment event cannot read future feedback, uncommitted evaluator truth, or a later carrier version. Each output receipt records the exact predecessor view.

Proposition 3.2 (Causal trace measurability)**.** *Under Assumption 3.1, every generated token, policy update, memory candidate, validation decision, commit decision, and pinned view is measurable with respect to the filtration at its event. Consequently a later version cannot be a parent of an earlier output in the declared structural trace.*

This statement excludes temporal leakage under the model; it does not prove that a real logging system timestamps events correctly.

3.3 Attempt-boundary state

Let $b(q, r)$ and $d(q, r)$ denote the begin and terminal event of attempt r. Define the attempt record

$$\mathcal{R}_{q,r} = \big(q, r, V_{q,r}, G_{b(q,r)}, Z_{q,r}, F_{q,r}, E^{\Phi}_{q,r}, E^{M}_{q,r}, d(q, r)\big), \tag{9}$$

where $E^{\Phi}_{q,r}$ and $E^{M}_{q,r}$ are ordered lists of carrier-specific events. An empty list is meaningful: it proves that no update of that carrier was admitted during the interval. A missing list is not equivalent to an empty list.

DESIGN PRESCRIPTION A conforming trace must preserve all attempt records, carrier ledgers, pin receipts, reset receipts, and sampler state. The public answer can remain concise; the audit need not store unrestricted private reasoning text.

3.4 Interleaving table

Order	Event	Reads	May write	Required receipt
1	PIN	latest authorized versions	view tuple only	both version/lineage IDs and compatibility result
2	GEN	pinned view, workspace, sampler	output and sampler trace	prompt/input hash, seed, decoding contract
3	FDBK	output, evaluator-visible outcome	feedback record	source, timing, scope, admissibility
4a	PUPD	pinned Φ, feedback, optimizer	policy carrier only	objective, gradient/update, predecessor/successor
4b	CAND/MCOM	pinned M, authorized evidence	memory carrier only after validation	complete candidate, validator, commit/NULL
5	PIN	independently current carriers	next view tuple	compatibility and exact versions

Table 2: One admissible partial order. Events 4a and 4b are not assumed to commute.

4 Independent Operation Algebras

4.1 Carrier-specific maps

Let $U_e : \mathcal{P} \times \mathcal{O} \to \mathcal{P} \times \mathcal{O}$ be an authorized policy update and $K_e : \mathcal{M} \to \mathcal{M}$ an authorized complete-row memory transition. Their lifted maps on the full state are

$$\widetilde{U}_e(W, P, A, C, G, Q) = (W, U_e(P), A, C, G, Q'), \tag{10}$$

$$\widetilde{K}_e(W, P, A, C, G, Q) = (W, P, K_e(A), C, G, Q''), \tag{11}$$

where Q' and Q'' append disjoint receipts. A coordinator event may later evaluate $\mathrm{Compat}(P, A, C)$ and refuse a new pin. It does not retroactively rewrite either carrier.

DEFINITION

Definition 4.1 (Operation algebras)**.** $\mathfrak{O}_\Phi$ contains policy snapshot, update, freeze, reset, swap, and rollback maps. $\mathfrak{O}_M$ contains memory snapshot, complete-row commit, exact NULL, reset, swap, row rollback, full-snapshot rollback, archive, and verified deletion maps. Each operation is closed over its own carrier type and appends to its own ledger.

4.2 Commutation is conditional

The carrier writes are disjoint, but their *inputs* can depend on a shared workspace or on the other carrier. Disjoint destination fields alone do not imply semantic commutation.

Assumption 4.2 (Input stability for a pair of events)**.** For a policy update U and memory transition K proposed from state S, (i) U's typed input and authorization decision are invariant to applying K first; (ii) K's candidate, validation, and authorization decision are invariant to applying U first; (iii) receipt append order is normalized without deleting event time; and (iv) both successor pairs satisfy the same compatibility predicate.

PROVED UNDER STATED ASSUMPTIONS

Theorem 4.3 (Conditional commutation)**.** *Under Assumptions 2.4 and 4.2, the carrier projection of the two-event successor is order-invariant:*

$$\pi_{P,A}(\widetilde{K} \circ \widetilde{U}(S)) = \pi_{P,A}(\widetilde{U} \circ \widetilde{K}(S)). \tag{12}$$

The full audit trace remains ordered and therefore need not be byte-identical.

Counterexample 4.4 (Disjoint writes, noncommuting decisions)**.** Suppose the memory candidate is admitted only if the current policy assigns it confidence above $1/2$. From Φ_0, confidence is 0.4 and K returns NULL. Feedback update U raises confidence to 0.6. Then $K \circ U$ commits a row whereas $U \circ K$ leaves memory unchanged, despite disjoint write privileges. Reversing the dependency—letting the policy loss read newly committed memory—gives the symmetric failure.

The counterexample is theoretical. It does not instantiate predictive admission or show that confidence is a valid admission score.

4.3 Compatibility without joint ownership

The coordinator may require

$$\mathrm{Compat}(P, A, C) = \mathbf{1}\{\text{schema, tenant, base-model, tokenizer, and feature contracts match}\}. \tag{13}$$

If compatibility fails, the only authorized action is to withhold a new view or route to a declared fallback. The coordinator must not silently reset P, rewrite A, or manufacture a composite version number.

Proposition 4.5 (No composite-version implication)**.** *Knowledge of a pinned pair (v^{Φ}, v^{M}) and a passing compatibility receipt does not determine either predecessor, lifetime, or rollback target. In particular, there exist two valid histories with the same pinned pair but different policy lineages, and two with the same pair but different memory lineages.*

The proof is by constructing independent fork-and-rejoin histories in Appendix A. A composite snapshot can be defined, but only as an explicit manifest containing two independently verifiable snapshots and a coordination receipt.

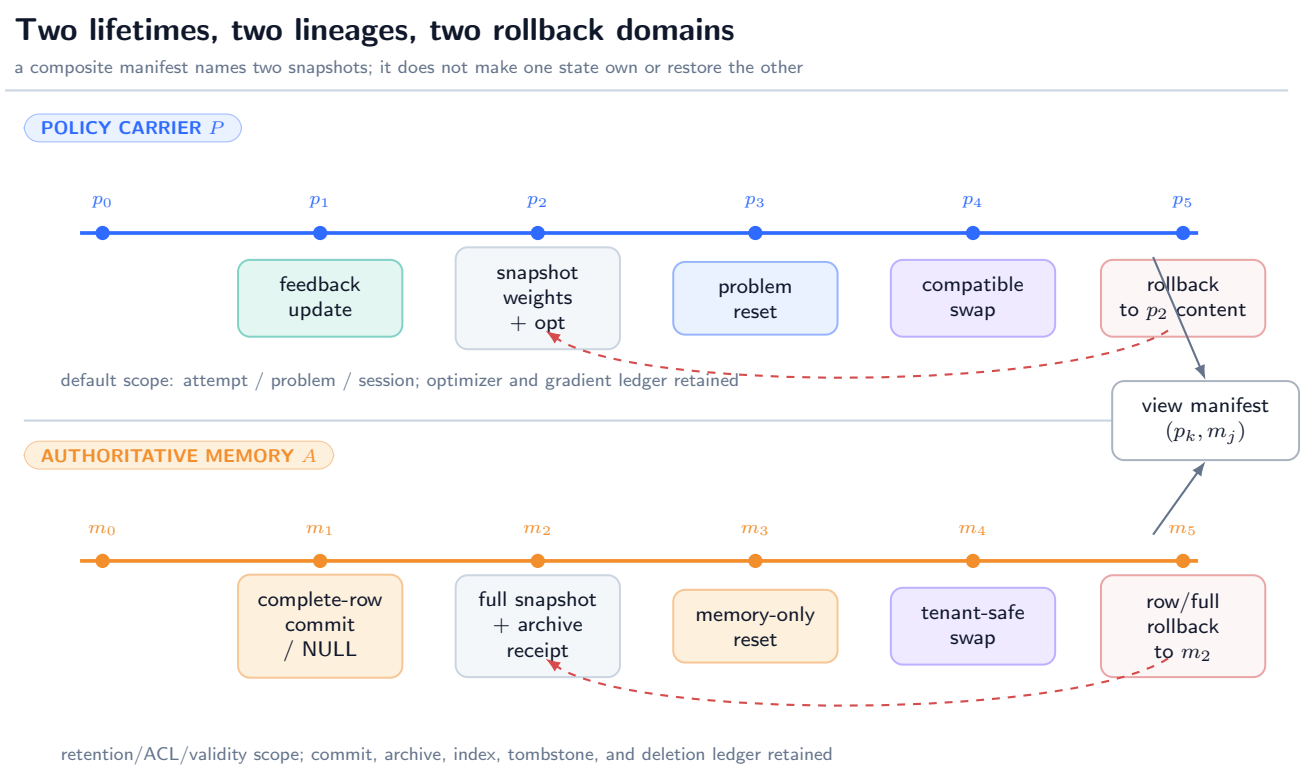

Figure 4: **Independent lifetimes, snapshots, rollback domains, and ledgers (DESIGN PRESCRIPTION).** A view manifest can name both carrier versions; it cannot collapse their lineages or authorize silent cross-restoration.

5 Strict RTT, Memory Adaptation, and Composition

5.1 A path-based definition

Let $Q_q(r) = 1$ mean that problem q remains unresolved immediately after attempt r. Let $F_{q,r}$ be admissible feedback and let $\Phi^{\text{read}}_{q,r}$ denote the policy version pinned by attempt r.

DEFINITION

Definition 5.1 (Strict reasoning-time learning)**.** An execution exhibits strict reasoning-time learning on problem q between attempts r and $s > r$ if all of the following hold:

1. $Q_q(k) = 1$ for every $k = r, \ldots, s-1$;
2. an authorized update event writes $\Phi^+ = U(\Phi^-, F_{q,r}, \eta)$ after feedback from attempt r;
3. a later attempt s pins and reads a descendant of Φ^+;
4. a registered intervention on that policy lineage can change the distribution of the later attempt while holding the declared non-policy state fixed.

The update has default problem scope unless a broader lifetime is explicitly authorized.

Conditions 1–3 establish timing and reuse; condition 4 turns a logged association into a causal mechanism. Merely computing a gradient, saving an adapter after the problem is solved, or updating on one problem and using the result only on unrelated future problems does not satisfy the strict within-problem definition used here.

DEFINITION

Definition 5.2 (Reasoning-time memory adaptation)**.** An execution exhibits reasoning-time memory adaptation (RTMA) when an authorized event changes M during an unresolved problem and a later attempt can read the new authoritative version. RTMA does not require a policy update and is not, by itself, strict RTT.

DEFINITION

Definition 5.3 (Dual-state composition)**.** A dual-state composition is an execution interval containing both a strict-RTT policy path and an RTMA path, with independently identifiable interventions and receipts. The definition does not require either main effect or their interaction to be beneficial.

5.2 Classification theorem

Assumption 5.4 (Attempt and carrier observability)**.** The audit exposes problem identity and unresolved status, event order, feedback source, carrier tags, predecessor and successor lineages, pinned versions, and intervention compliance. Missing receipts are represented as missing, not imputed as no-ops.

PROVED UNDER STATED ASSUMPTIONS

Theorem 5.5 (Mechanism classification)**.** *Under Assumptions 3.1 and 5.4, an interval belongs to exactly one of the following four observable mechanism classes:*

	no later read of updated M	*later read of updated M*
no strict policy path	*static/retry*	*RTMA only*
strict policy path	*strict RTT only*	*dual-state composition*

provided that "path present" uses the definitions above. The classes say nothing about endpoint improvement.

Corollary 5.6 (Memory mutation is insufficient for strict RTT)**.** *If Φ is frozen throughout an interval and no policy descendant is read by a later same-problem attempt, any change caused solely by M belongs to RTMA only, not strict RTT.*

Counterexample 5.7 (Retry masquerading as learning)**.** Two attempts use identical Φ, identical M, and different sampler seeds. The second succeeds. The output improvement is compatible with pure sampling variation and therefore certifies neither RTT nor RTMA.

Counterexample 5.8 (Post-resolution adaptation)**.** Attempt r solves the problem, after which feedback updates Φ for a future task. This may be online or continual adaptation, but it is not strict reasoning-time learning for the resolved problem under Definition 5.1.

5.3 Mechanism receipts

Claimed mechanism	Minimum positive receipt	Required negative control
Strict RTT	same problem, unresolved marker, feedback, Φ predecessor/successor, later pinned descendant	reset/fresh/frozen or swapped Φ with M and sampler held fixed
RTMA	complete-row commit/NULL, M predecessor/successor, later pinned descendant	memory reset/swap/freeze with Φ and sampler held fixed
Dual-state composition	both receipt chains plus exact interleaving	independent 2×2 interventions and compliance audit
Benefit	preregistered endpoint under randomized intervention	seed, order, leakage, and attrition checks

Table 3: A mechanism receipt is necessary but not an outcome result.

DESIGN PRESCRIPTION Names such as "test-time learning," "memory," or "self-improvement" are never compliance evidence. The trace and interventions determine classification.

6 Reset, Swap, Snapshot, and Rollback Semantics

6.1 Independent operations

For carrier $X \in \{\Phi, M\}$, write $\text{Snapshot}_X(h)$ for a content-addressed snapshot at history position h, $\text{Reset}_X(b)$ for reset to a declared baseline b, $\text{Swap}_X(s)$ for replacement by an independently sourced compatible snapshot, and $\text{Rollback}_X(h)$ for restoration to a prior authorized snapshot with a new rollback receipt. Rollback never erases history; it creates a successor whose content may equal an earlier snapshot.

Assumption 6.1 (Carrier-local restoration)**.** Each restoration operation names exactly one carrier, verifies its target snapshot and lineage policy, changes only that carrier and its ledger, invalidates carrier-local caches, and appends a successor version. Cross-carrier cache invalidation or pin refusal may occur, but the other carrier's content and lineage remain unchanged unless a separate authorized event names it.

PROVED UNDER STATED ASSUMPTIONS

Theorem 6.2 (Independent rollback preservation)**.** *Under Assumptions 2.4 and 6.1, applying $\text{Rollback}_\Phi(h)$ leaves the content, version, and lineage of M unchanged; applying $\text{Rollback}_M(k)$ leaves the content, version, and lineage of Φ unchanged. A subsequent compatibility failure may block a pin but cannot silently restore the other carrier.*

Corollary 6.3 (Explicit composite recovery)**.** *Restoring both carriers requires two authorized carrier-local events or an explicit composite manifest that expands into those two events. A single unnamed "system rollback" is nonconforming.*

6.2 Reset and swap controls

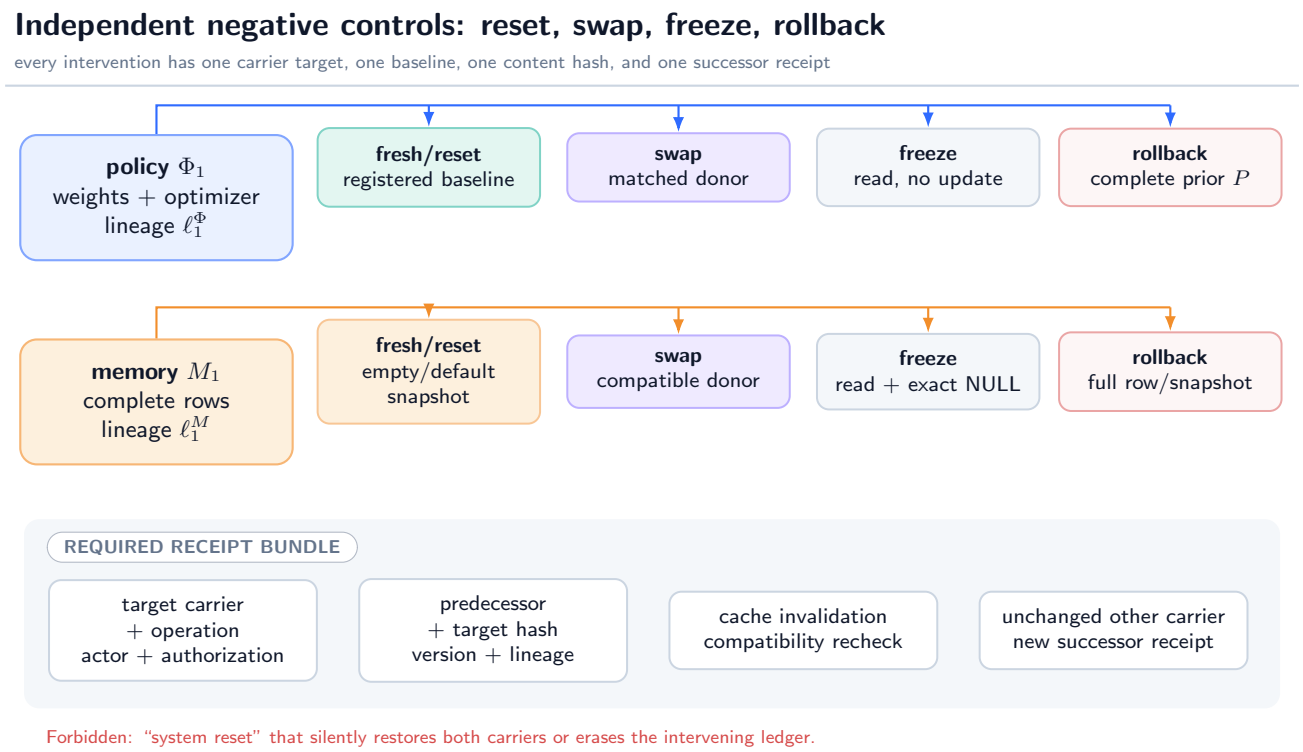

Figure 5: **Reset, swap, freeze, and rollback controls with receipts (DESIGN PRESCRIPTION).** Each intervention targets one carrier. A composite control is two declared interventions, never an implicit cross-state restoration.

The controls answer different questions:

- fresh_Φ restores the registered initialization and optimizer state; reset_Φ may restore another declared problem baseline.
- swap_Φ imports a compatible policy snapshot from a matched unit, testing content specificity while preserving nominal update count.
- freeze_Φ permits reads but blocks updates, separating extra attempts from adaptation.
- $\mathsf{rollback}_\Phi$ tests recovery to a known policy snapshot and must include optimizer state unless the protocol explicitly studies optimizer mismatch.
- fresh_M installs the registered empty/default authoritative state; reset_M restores a specified authoritative snapshot.
- swap_M imports a compatible memory snapshot from a matched unit, testing whether success depends on problem-specific authoritative content.
- freeze_M permits reads and exact NULL only; $\mathsf{rollback}_M$ restores a full row version or full snapshot without changing Φ.

Operation	Target content	Required companion state	Invalid shortcut
Reset_Φ	parameters and declared optimizer baseline	same M, context, sampler contract	zeroing weights while keeping stale moments
Swap_Φ	compatible independent policy snapshot	same M and view constraints	changing base model/tokenizer with the swap
$\mathsf{Rollback}_\Phi$	prior policy checkpoint plus lineage	unchanged M	rewriting history or memory to "match"
Reset_M	exact registered memory baseline	same Φ	clearing only an index while rows remain
Swap_M	compatible independent full snapshot	same Φ	importing rows without tenant/provenance checks
$\mathsf{Rollback}_M$	full row or full snapshot	unchanged Φ	fieldwise partial undo or silent policy restoration

Table 4: Carrier-specific recovery semantics.

6.3 Lineage and retention

Let $L_X = (V_X, E_X)$ be the directed acyclic lineage graph for carrier X. Every event edge records predecessor, successor, operation, actor, timestamp/order, input hashes, authorization, and outcome. A rollback points to an earlier content snapshot but creates a new vertex. A deletion/tombstone event in L_M follows the memory retention contract; it does not delete a policy receipt. A policy reset in L_Φ cannot shorten an authoritative row's retention lifetime.

Proposition 6.4 (No silent history erasure)**.** *If lineage vertices and edge receipts are append-only and content-addressed, then a restore operation cannot make its existence observationally identical to an uninterrupted continuation for an auditor who reads the ledger, even when restored content equals an earlier snapshot.*

This is an auditability statement under append-only storage, not a guarantee against a privileged ledger compromise.

7 The Compress-All Comparator

7.1 Why a comparator needs assumptions

Recurrent and test-time-learning systems often compress observations into a bounded numerical state [6, 2, 8, 1]. Compression is not inherently defective. The relevant question is whether indiscriminately mixing a stream into one trainable carrier causes more harmful cross-event influence than a typed, gated design for a declared loss.

Let observations be $X_1, \ldots, X_T$, with informative indices I and contaminants C. A projected compress-all update is

$$\Phi_{t+1} = \mathrm{proj}_{\mathcal{P}}\{\Phi_t - \eta_t g(\Phi_t, X_t)\}. \tag{14}$$

Let Φ_t^I be the coupled trajectory that replaces $g(\Phi_t, X_t)$ by zero for $t \in C$ while sharing initialization and informative inputs.

Assumption 7.1 (Conditional update sensitivity)**.** Projection is nonexpansive; for every x, $g(\cdot, x)$ is L_g-Lipschitz; and contaminant updates satisfy $\|g(\Phi_t^I, X_t)\| \le B_t$ for $t \in C$. The downstream loss $\ell(\Phi)$ is L_ℓ-Lipschitz on $\mathcal{P}$.

PROVED UNDER STATED ASSUMPTIONS

Theorem 7.2 (Conditional contamination bound)**.** *Under Assumption 7.1,*

$$\|\Phi_T - \Phi_T^I\| \le \sum_{t \in C} \eta_t B_t \prod_{u=t+1}^{T-1} (1 + \eta_u L_g), \tag{15}$$

$$|\ell(\Phi_T) - \ell(\Phi_T^I)| \le L_\ell \sum_{t \in C} \eta_t B_t \prod_{u=t+1}^{T-1} (1 + \eta_u L_g). \tag{16}$$

The bound is one-sided only if an additional sign condition is supplied; by itself it bounds magnitude, not harm.

The product exposes amplification by later update sensitivity. It is not a measured contamination rate and does not compare hardware efficiency.

7.2 When compression is optimal

Counterexample 7.3 (Sufficient-statistic optimum)**.** Let $X_t \mid \theta \sim \mathcal{N}(\theta, \sigma^2)$ independently with a Gaussian prior on θ, and let the next prediction minimize squared error. The posterior is summarized exactly by a bounded pair consisting of precision and precision-weighted mean. Updating that pair with every observation is Bayes-optimal. A second authoritative row store cannot improve Bayes risk when it adds no information beyond the sufficient statistic and costs are nonnegative.

Counterexample 7.4 (Contamination can help)**.** If indices labeled "contaminants" are actually informative under the target distribution, suppressing their updates increases risk. The sign of Equation (16) cannot be inferred from a taxonomy chosen after observing outcomes.

Proposition 7.5 (No universal dominance)**.** *Without restrictions on the data law, state capacity, update class, downstream loss, and resource cost, neither a dual-state architecture nor compress-all recurrence uniformly dominates the other.*

Thus this paper makes no universal attack on recurrent models, state-space models, TTT layers, or neural long-term memory. Its claim is narrower: if a system asserts authoritative semantics and policy adaptation as distinct mechanisms, then it must expose the distinctions and controls required here.

7.3 Typed gating as a conditional intervention

Let $A_t \in \{0, 1\}$ be a preregistered gate deciding whether event t contributes to Φ. The gated path uses $A_t g(\Phi_t, X_t)$. If A_t depends on hidden future outcomes, comparisons are confounded. If it depends on causal features and is randomized or otherwise identified, its contribution can be studied. This definition is not predictive memory admission: A_t governs a policy update, not a complete authoritative row commit.

PLANNED TEST A future comparator must register the observation taxonomy, target law, update budget, parameter count, context budget, evaluation loss, and all system costs. Calling one carrier "compressed" and another "memory" does not equalize capacity or information.

8 Policy Drift and Dual-State Interaction Bounds

8.1 Bounded policy drift

Consider K authorized policy updates

$$\Phi_{k+1} = \mathrm{proj}_{\mathcal{P}}(\Phi_k - \eta_k \widehat{g}_k), \qquad \|\widehat{g}_k\| \le G_k. \tag{17}$$

PROVED UNDER STATED ASSUMPTIONS

Proposition 8.1 (Path-length bound)**.** *If projection fixes every $\Phi_k \in \mathcal{P}$ and is nonexpansive, then*

$$\|\Phi_K - \Phi_0\| \le \sum_{k=0}^{K-1} \eta_k G_k. \tag{18}$$

If the output distribution satisfies $\mathrm{TV}(p_\Phi(\cdot \mid w, m), p_{\Phi'}(\cdot \mid w, m)) \le L_p \|\Phi - \Phi'\|$, its drift is at most $L_p \sum_k \eta_k G_k$ at fixed workspace and memory.

The result controls displacement, not improvement, stability to distribution shift, or safety. A zero gradient also satisfies the bound and learns nothing.

8.2 Interaction-sensitive error

Let the deployed predictor be $h(\Phi, M, W)$. Fix reference states (Φ_0, M_0) and define perturbations $\delta_\Phi = d_\Phi(\Phi, \Phi_0)$ and $\delta_M = d_M(M, M_0)$. Suppose a scalar loss obeys

$$|\ell(\Phi, M) - \ell(\Phi_0, M_0)| \le L_\Phi \delta_\Phi + L_M \delta_M + L_{\Phi M} \delta_\Phi \delta_M. \tag{19}$$

The cross term permits one carrier's displacement to amplify sensitivity to the other. It does not imply complementarity; the bound is unsigned.

Assumption 8.2 (Local mixed sensitivity)**.** Equation (19) holds on the registered intervention region, and the state metrics, reference states, and constants are fixed before evaluation.

PROVED UNDER STATED ASSUMPTIONS

Theorem 8.3 (Dual-state perturbation envelope)**.** *Under Assumption 8.2, if interventions guarantee $\delta_\Phi \le \epsilon_\Phi$ and $\delta_M \le \epsilon_M$, then every unit's absolute loss displacement is bounded by*

$$B_{\Phi M} = L_\Phi \epsilon_\Phi + L_M \epsilon_M + L_{\Phi M} \epsilon_\Phi \epsilon_M. \tag{20}$$

For a bounded loss, the same envelope bounds the absolute population mean displacement. No sign or causal decomposition follows without factorial interventions.

8.3 Four interaction regimes

For losses, define unit-level main effects $\tau_\Phi = Y_{10} - Y_{00}$, $\tau_M = Y_{01} - Y_{00}$, and interaction $\tau_{\Phi M} = Y_{11} - Y_{10} - Y_{01} + Y_{00}$.

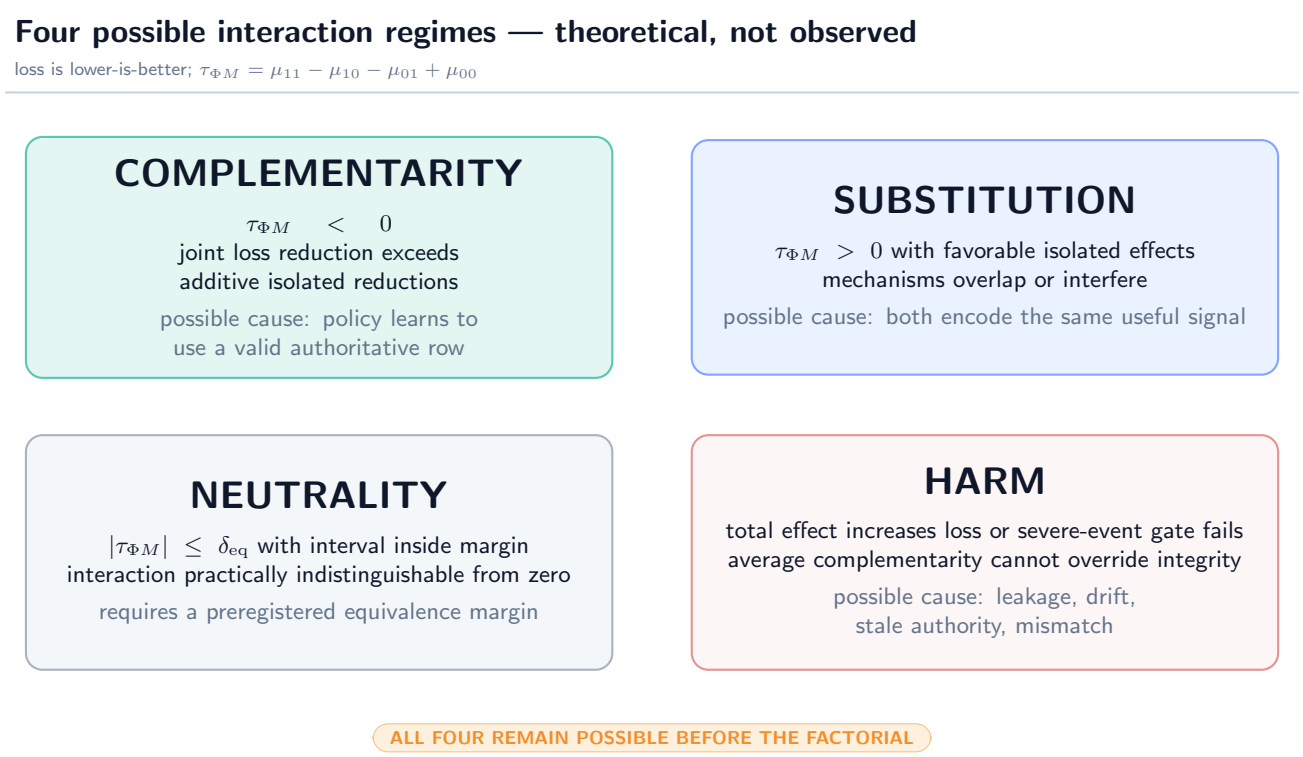

Figure 6: **Theoretical/planned interaction regimes** (DEFINITION / PLANNED TEST)**.** Complementarity, substitution, neutrality, and harm are signs of registered contrasts, not observed dual-state results.

- *Complementarity*: $\mathbb{E}[\tau_{\Phi M}] < 0$; the joint loss reduction exceeds the sum of isolated reductions.
- *Substitution*: isolated effects improve loss but $\mathbb{E}[\tau_{\Phi M}] > 0$; the mechanisms overlap or interfere.
- *Neutrality*: the interaction is practically indistinguishable from zero under a preregistered equivalence margin.
- *Harm*: at least one registered total effect increases loss or violates a severe-event constraint, regardless of average interaction.

Counterexample 8.4 (Average complementarity hides a severe subgroup)**.** Nine units improve jointly by one loss unit and one protected subgroup unit worsens by eight. The mean joint effect is favorable, but a preregistered severe-event or subgroup gate can still fail. Average interaction is not a safety certificate.

DESIGN PRESCRIPTION The future protocol must report all four cell means, main effects, interaction, uncertainty, required subgroups, severe outcomes, and total cost. It may not display only the best arm.

9 The Two-by-Two Policy-by-Memory Factorial

9.1 Assignments, states, and potential outcomes

For experimental unit i (a problem-group, not an individual attempt), let $Z_i^{\Phi}, Z_i^{M} \in \{0,1\}$ be independently randomized assignments. Assignment 1 means the registered updated carrier is made available; assignment 0 means the registered baseline/fresh carrier. Let D_i^{Φ}, D_i^{M} be realized compliance indicators and $Y_i(p,m)$ the loss that would be observed under realized carrier interventions (p,m). This potential-outcome notation follows the distinction between assignments and outcomes [7].

DEFINITION

Definition 9.1 (Factorial estimands)**.** For $p,m \in \{0,1\}$ define $\mu_{pm} = \mathbb{E}[Y_i(p,m)]$ and

$$\tau_{\Phi|m} = \mu_{1m} - \mu_{0m}, \tag{21}$$

$$\tau_{M|p} = \mu_{p1} - \mu_{p0}, \tag{22}$$

$$\tau_{\Phi M} = \mu_{11} - \mu_{10} - \mu_{01} + \mu_{00}, \tag{23}$$

$$\bar{\tau}_{\Phi} = \tfrac{1}{2}(\tau_{\Phi|0} + \tau_{\Phi|1}), \qquad \bar{\tau}_{M} = \tfrac{1}{2}(\tau_{M|0} + \tau_{M|1}). \tag{24}$$

Negative effects improve a loss endpoint. Every endpoint and materiality margin is preregistered.

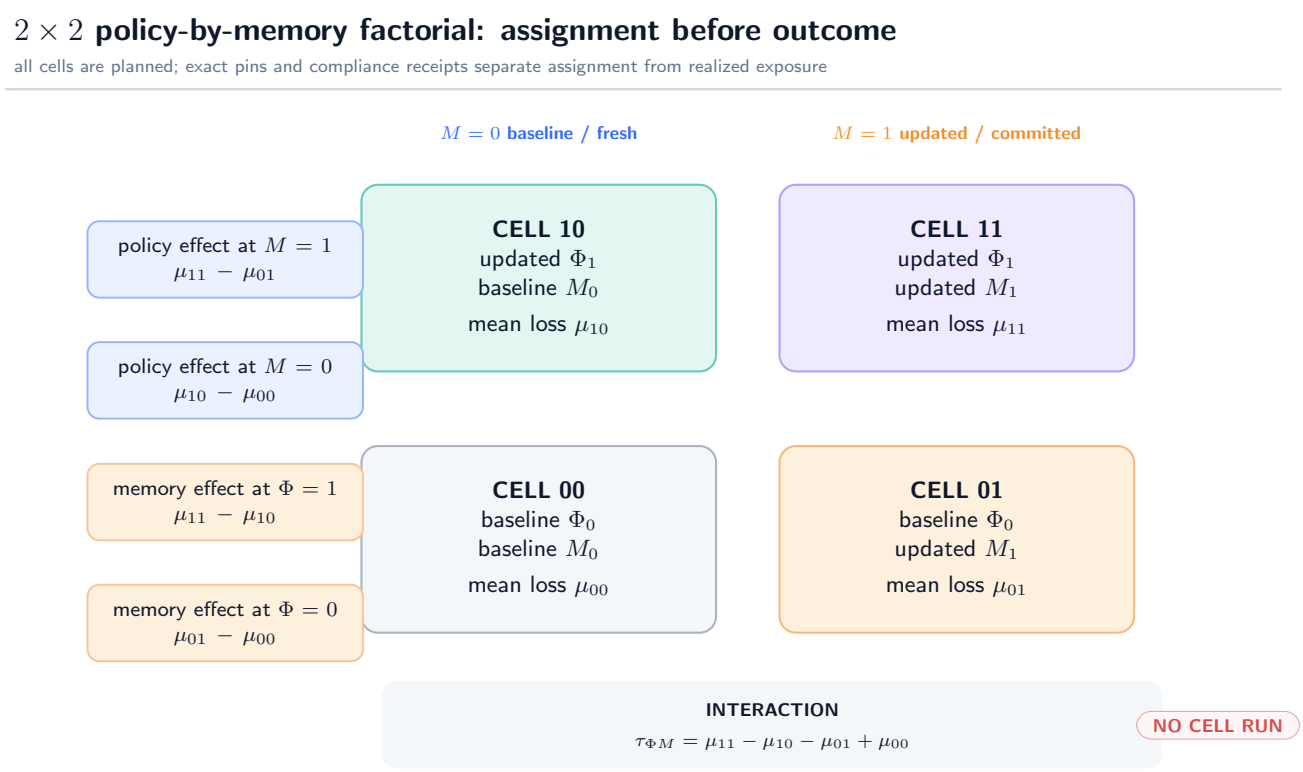

Figure 7: **Independent 2×2 interventions and contrasts (PLANNED TEST).** Each cell pins its assigned carrier versions while holding base model, visible workspace, sampler contract, and resource budget to the registered design. No cell has been run.

Assumption 9.2 (Factorial identification)**.** Units are assigned with known positive probability to all four cells; assignments precede outcome generation; consistency and no interference hold at the registered group boundary; carrier versions are pinned before the evaluated attempt; attrition and endpoint construction do not depend on hidden potential outcomes; and the analysis either has perfect compliance or distinguishes assignment from receipt-verified exposure.

PROVED UNDER STATED ASSUMPTIONS

Theorem 9.3 (Identification under randomized compliant intervention)**.** *Under Assumption 9.2 with perfect compliance,*

$$\mathbb{E}[Y_i^{\text{obs}} \mid Z_i^{\Phi} = p, Z_i^{M} = m] = \mu_{pm}. \tag{25}$$

Therefore cell-mean differences identify both conditional main effects, both marginal main effects, and $\tau_{\Phi M}$. With known unequal assignment probabilities, Horvitz–Thompson cell means are unbiased under the same premises.

The theorem does not establish exclusion restrictions under noncompliance, adequate power, metric validity, or any beneficial sign.

9.2 Fresh, reset, swapped, and frozen variants

The primary binary factor must be defined exactly. A useful staged design is:

1. *availability factorial*: updated versus registered fresh baseline for each carrier;
2. *content-specificity controls*: problem-matched updated carrier versus compatible swapped carrier;
3. *attempt-budget control*: mutable versus frozen policy with identical extra attempts;
4. *recovery controls*: independently roll back one carrier after a registered checkpoint and test restoration of its isolated effect;
5. *order control*: randomize admissible update/commit order where both are enabled.

Each stage uses new held-out groups or a registered repeated-measures design that models carryover. A carrier called "reset" without a content hash, lineage, optimizer state (for Φ), and cache invalidation receipt is noncompliant.

9.3 Noncompliance and exclusions

If $D \neq Z$, the intent-to-treat contrast remains identified by random assignment, but it estimates assignment, not carrier exposure. A complier effect would require additional monotonicity and exclusion assumptions that are especially fragile when pin failures change latency or fallback behavior. This paper does not assume those conditions.

Proposition 9.4 (Receipt-stratified descriptive effect is not automatically causal)**.** *Conditioning on successful pin or commit after randomization can select on post-assignment variables. Unless compliance is guaranteed by design or modeled under explicit assumptions, the difference between receipt-positive units is descriptive rather than the randomized cell effect.*

PLANNED TEST The protocol in Appendix B uses intent-to-treat as primary, reports compliance separately, and stops rather than silently dropping failed pins, validator rejects, timeouts, or exact NULL outcomes.

10 Observational Non-Identification

10.1 Output traces do not name the changed carrier

Let the observable trace be $O = (q, Z_0, F_0, Z_1)$, omitting internal state and receipts. The following impossibility result explains why a successful retry cannot classify the mechanism.

PROVED UNDER STATED ASSUMPTIONS

Theorem 10.1 (Output-trace equivalence)**.** *For any finite observable law over O generated by a system whose later output changes through a policy-state update, there exists a system with frozen Φ and a memory-state update that induces the same observable law. Conversely, there exists a frozen-memory policy-update system matching any such memory-update observable law, provided the alternate carrier has sufficient finite capacity to encode the later-output kernel. Therefore O alone does not identify which carrier changed.*

The construction does not say that the two implementations share cost, privileges, interpretability, or semantics. It shows only observational equivalence at the chosen output interface.

Corollary 10.2 (No interaction from one trajectory)**.** *A single dual-state trajectory cannot identify $\tau_{\Phi M}$ because at most one of the four potential outcomes is observed for its experimental unit.*

10.2 Hidden shared state

Even carrier hashes are insufficient if a third state changes. Let G be sampler state and Q a cache. Two runs with equal (Φ, M) but different G can have different outputs. Two runs with equal carrier payloads but a stale cache in Q can read different effective memory. Hence the intervention must pin or log the complete state inventory from Equation (6).

Counterexample 10.3 (Cache-mediated false memory effect)**.** Assignment changes M, but the inference process continues reading a cached old row. The intended memory intervention has no exposure, while a latency-triggered retry changes the sampler. An analysis using assigned memory labels attributes the output difference to M. Pin, cache-invalidation, and read receipts would expose the violation.

Counterexample 10.4 (Feedback-dependent stopping)**.** The updated-policy arm is allowed more attempts only after poor feedback. Comparing final accuracy to a fixed-attempt baseline mixes policy state with an adaptive compute budget. The policy factor must define or condition on a registered stopping rule without post-treatment cherry-picking.

10.3 Swap tests and content specificity

A successful updated-versus-fresh comparison can still reflect update count, file age, cache warming, or lineage-specific configuration. A compatible swap control tests whether the actual state content matters. Let S_i^{Φ} be another unit's policy snapshot and S_i^{M} another unit's memory snapshot, matched on schema, update count, resource budget, and non-content metadata. If the updated state helps but its swapped counterpart helps equally, problem-specific content has not been isolated.

Assumption 10.5 (Valid swap)**.** Swap donors are selected independently of recipient potential outcomes; compatibility fields are identical; no recipient-specific content leaks into the donor; and all non-content execution differences are balanced or fixed.

PROVED UNDER STATED ASSUMPTIONS

Proposition 10.6 (Swap contrast interpretation)**.** *Under Assumption 10.5 and randomized assignment, the updated-versus-swapped contrast identifies the effect of recipient-specific carrier content relative to the declared donor distribution, not a universal semantic-content effect.*

10.4 Missing provenance

If predecessor hashes, carrier tags, or pin receipts are missing, multiple incompatible histories fit the same terminal bytes. No statistical estimator can recover a uniquely identified reset, rollback, or update lineage from terminal state alone without additional structural restrictions.

DESIGN PRESCRIPTION A missing receipt is a protocol failure. It is never converted into "no update," "successful reset," or "same memory."

11 Authorization and Information-Flow Isolation

11.1 Separate writers and admissible inputs

Policy feedback can contain scalar rewards, verifier outcomes, public error classes, or other registered signals. It must not automatically become authoritative memory content. A memory candidate can contain a canonical fact and provenance but must not be interpreted as a gradient. The two flows can originate from one observed event while remaining separately authorized.

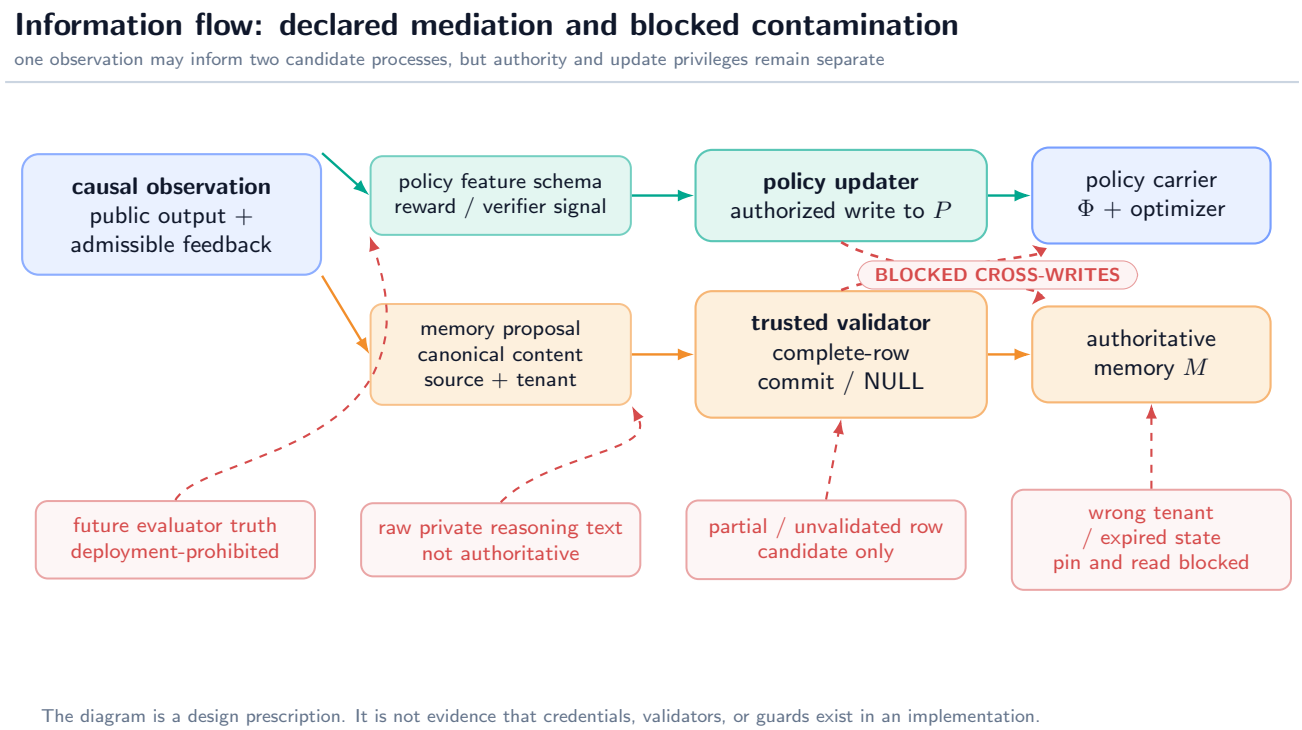

Figure 8: **Contamination pathways and blocked privilege crossings (DESIGN PRESCRIPTION).** Teal arrows are declared reads or typed updates. Coral dashed arrows are forbidden: gradients cannot write authoritative rows; unvalidated candidates cannot alter policy; private reasoning text is not authority; future evaluator truth cannot enter deployment.

Assumption 11.1 (Least-privilege mediation)**.** Policy and memory writers execute under distinct credentials. Candidate construction is non-authoritative. Only the trusted memory validator/committer can publish a complete row or exact NULL. Only the policy updater can publish a policy successor. Every cross-carrier feature is explicitly serialized, schema-checked, and logged.

PROVED UNDER STATED ASSUMPTIONS

Proposition 11.2 (Blocked direct contamination)**.** *Under Assumptions 2.4 and 11.1, a compromised ordinary policy-update call cannot directly publish an authoritative memory successor, and a memory candidate cannot directly publish a policy successor. The statement does not cover compromise of both privileged services or a shared trusted computing base.*

11.2 Trusted assembly and exact NULL

The inherited row contract requires a complete target-conditioned row with canonical and neural views, slot, tenant and ACL fields, monotone version, validity, protection, provenance, rollback pointer, deletion/tombstone semantics, and a consistency receipt [10]. A neural module may propose semantic content, but trusted assembly owns protected fields and validation. If any required field or receipt fails, the authoritative transition is exact NULL: resident bytes and authoritative version do not change, while a rejection receipt is appended outside the resident state.

UNVALIDATED This paper uses that contract as a formal dependency only. The same-checkpoint semantic interface remains failed/unqualified; no dual-state row was generated, admitted, or read by a model.

11.3 Feedback-to-memory leakage

Suppose an evaluator exposes the correct final answer after attempt r. Using that answer as policy feedback may be allowed in a registered supervised test-time setting. Writing it into M and scoring attempt $r + 1$ on the same answer tests answer

caching, not general reasoning or predictive memory. The memory protocol must tag target-revealing feedback and exclude or separately analyze direct answer reuse.

Counterexample 11.3 (Joint endpoint leakage)**.** Both Φ and M receive the ground-truth answer. The joint arm succeeds perfectly on a repeated query. The factorial interaction may appear strongly complementary even though both carriers are merely copying target information. Randomization does not repair an invalid endpoint or post-treatment leakage.

11.4 Tenant, lifetime, and deletion boundaries

Policy state defaults to the current problem and must be destroyed or archived according to its own scope. Memory rows follow tenant, ACL, validity, protection, retention, tombstone, and verified-deletion rules. A policy snapshot cannot extend a memory row's retention; a memory archive cannot preserve an expired problem-scoped optimizer state by reference. Composite manifests hold identifiers, not ownership of the underlying retention policy.

Boundary	Required behavior	Stop condition
Carrier privilege	distinct credentials and typed write endpoints	any cross-carrier direct write
Future information	deployment reads causal data only	evaluator truth or future branch enters a deployed update
Authority	candidate remains non-authoritative until full validation	partial row, missing protected field, or hash-only equality
Tenant/ACL	check at construction, pin, read, commit, archive, and restore	mismatched tenant or unverified policy
Lifetime	independently declared expiry and cleanup	one carrier silently inherits the other's lifetime
Deletion	carrier-local verified deletion/tombstone receipt	dangling index, cache, archive, or snapshot

Table 5: Information-flow and authority gates.

12 Complete Resource Accounting

12.1 Vector costs before scalarization

For execution u, define

$$\mathbf{C}(u) = (N_{\text{tok}}, F_{\text{fwd}}, F_{\text{bwd}}, B_{\text{param}}, B_{\text{opt}}, B_{\text{row}}, B_{\text{archive}}, B_{\text{traffic}}, T_{\text{wall}}, E, N_{\text{retry}}, N_{\text{receipt}}). \tag{26}$$

The entries are tokens, forward and backward work, parameter bytes, optimizer bytes, resident-row bytes, archive/index bytes, memory/network traffic, wall time, energy, retries, and receipt/validation operations. A scalar cost $c = \lambda^{\top}\mathbf{C}$ is reported only with a preregistered nonnegative weight vector λ and units.

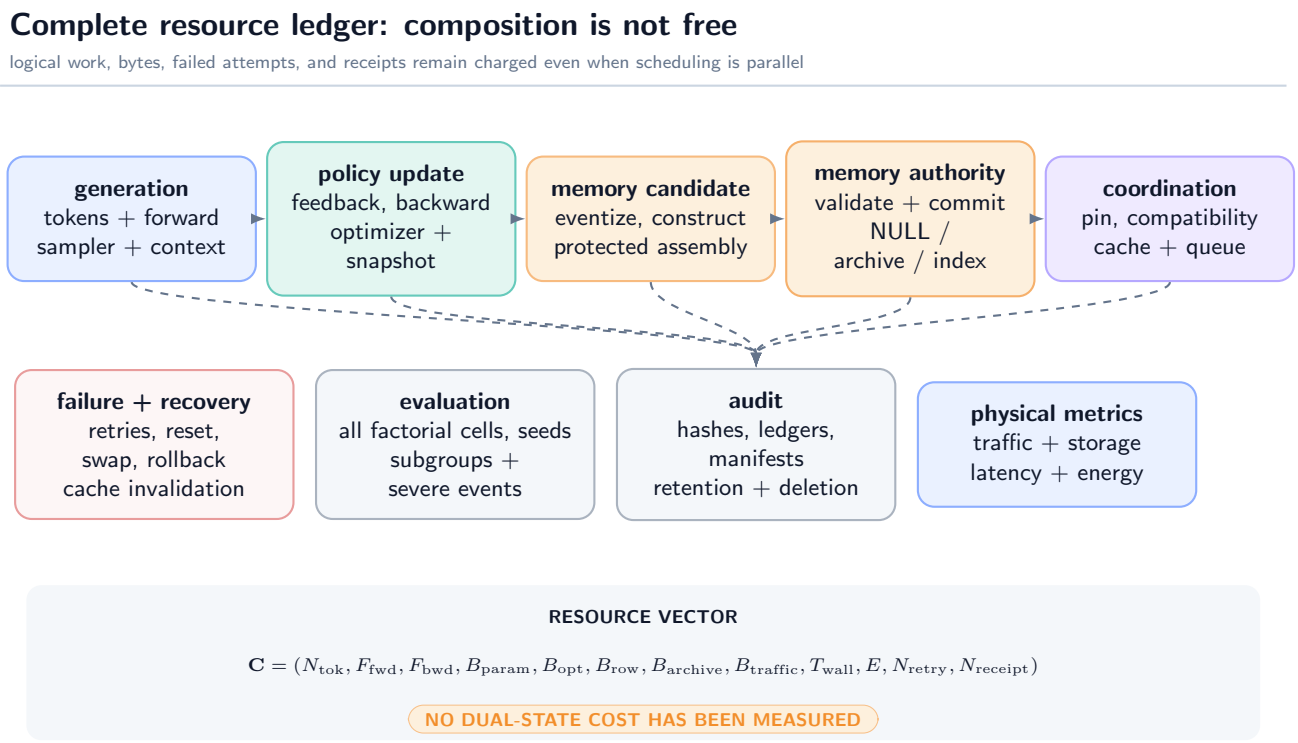

Figure 9: **Training, reasoning-time, memory, recovery, and audit ledgers (DEFINITION / DESIGN PRESCRIPTION).** Parallelism may reduce wall time but cannot delete logical work, bytes, validation, or failed attempts from the ledger.

12.2 Ledger decomposition

Let $\mathcal{E}_{\text{gen}}$, $\mathcal{E}_{\Phi}$, $\mathcal{E}_{M}$, $\mathcal{E}_{\text{coord}}$, and $\mathcal{E}_{\text{recover}}$ be disjoint event sets. Exact accounting gives

$$\mathbf{C}_{\text{total}} = \sum_{e \in \mathcal{E}_{\text{gen}}} \mathbf{c}_e + \sum_{e \in \mathcal{E}_{\Phi}} \mathbf{c}_e + \sum_{e \in \mathcal{E}_{M}} \mathbf{c}_e + \sum_{e \in \mathcal{E}_{\text{coord}}} \mathbf{c}_e + \sum_{e \in \mathcal{E}_{\text{recover}}} \mathbf{c}_e. \tag{27}$$

PROVED UNDER STATED ASSUMPTIONS

Proposition 12.1 (No-free-composition accounting)**.** *If every physical or logical operation is assigned to exactly one event set, Equation (27) is an identity. In particular, dual-state cost is not represented by generation tokens alone: it includes feedback/evaluation, backward and optimizer work, checkpointing, complete candidate construction, protected-field assembly, validation, commit/NULL, archive/index traffic, pin/compatibility checks, resets, swaps, rollbacks, cache invalidation, retries, and audit receipts.*

Corollary 12.2 (Parallelism does not erase work)**.** *Batching or concurrent execution may change T_{wall} and hardware utilization but does not set the corresponding logical operation counts or transferred bytes to zero.*

12.3 Training and reasoning-time ledgers

Phase	Mandatory charges	Common hidden omission
Base/pretraining	data, tokens, forward/backward, optimizer, checkpoints	amortizing it for one arm but not another
Meta/policy preparation	task sampling, reward/evaluator, adapters, optimizer state, validation	counting only trainable parameter bytes
Reasoning-time policy	exploration attempts, feedback, reward, backward pass, optimizer, snapshots, rollback tests	reporting only final-answer tokens
Memory candidate	eventization, target-conditioned construction, canonical/neural views, protected-field assembly	treating the row as already available
Memory authority	validation, exact NULL decisions, commit, archive, index, provenance, deletion	counting resident bytes only
Coordination	version pin, compatibility, cache invalidation, queueing, fallback	assuming a free atomic pair
Evaluation	all four factorial cells, negative controls, seeds, subgroups, failures, uncertainty	reporting the best cell alone

Table 6: Complete logical ledger. No cost is measured in this paper.

12.4 Comparator normalization

A fair comparator holds fixed or explicitly prices base model, precision, maximum visible context, total generated tokens, number of attempts, policy parameter count, memory capacity, evaluator access, update budget, wall-clock cap, and hardware.

If architectures cannot be matched on all dimensions, the result is a Pareto comparison over quality, severe failures, latency, energy, memory, and traffic, not a single "efficiency" number.

Assumption 12.3 (Cost observability)**.** Every event reports units and measurement method; failed and aborted events are retained; cached reuse is proven by content identity and dependency validity; and energy or FLOP claims are omitted when the required measurement is absent.

PLANNED TEST No dual-state cost vector has been measured. Exact ledger equations are protocol accounting, not evidence of efficiency or a hardware crossover.

13 Failure Modes and Recovery

13.1 Carrier-local failures

Carrier	Failure	Observable symptom	Required response
Φ	divergent or nonfinite update	parameter/optimizer validation fails	reject successor; retain predecessor; record failed update
Φ	optimizer mismatch	weights restore but moments/step do not	quarantine; restore complete policy snapshot or stop
Φ	reward leakage/hacking	success depends on prohibited evaluator information	invalidate affected stage and descendants
Φ	excessive drift	registered norm, KL, or severe-output gate fails	freeze and independently roll back policy only
M	partial or malformed row	protected field, width, canonical/neural consistency, or hash check fails	exact NULL; no authoritative version change
M	stale/unauthorized authority	tenant, ACL, validity, or version check fails	block read/commit; rollback or tombstone under memory policy
M	archive/index divergence	resident, index, and archive receipts disagree	quarantine memory view; repair and revalidate
M	deletion failure	row survives in cache/index/archive	stop; complete verified deletion before reuse

Table 7: Carrier-local failures preserve separate recovery domains.

13.2 Composition failures

Composition failure	Why it breaks inference	Recovery/analysis consequence
Unpinned mixed read	tokens within one attempt can see different carrier versions	invalidate attempt; no post hoc selection of a convenient view
Compatibility mismatch	policy features, tokenizer, schema, or base model disagree	block pin or route to registered fallback; do not coerce bytes
Hidden joint reset	one intervention changes both carriers	factorial attribution fails; rerun from independently verified baselines
Undeclared event order	policy and memory decisions may not commute	effect is undefined for the registered treatment
Shared credential	a compromise can cross both write domains	least-privilege proposition does not apply
Receipt loss	predecessor, exposure, or compliance is ambiguous	mark missing; do not infer no-op or successful rollback
Feedback-to-answer leakage	later success can be direct target copying	invalidate reasoning endpoint or report separate cache task
Adaptive resource imbalance	one cell receives more attempts or compute	endpoint mixes mechanism and budget

Table 8: Composition failures and their causal consequences.

13.3 Recovery invariants

DESIGN PRESCRIPTION A recovery sequence must satisfy:

1. stop new pins that could read a suspect carrier;
2. preserve both ledgers and all failed events;
3. identify the smallest carrier-local recovery target;
4. restore that target with a new successor receipt;
5. invalidate carrier-dependent caches without changing the other carrier;
6. rerun compatibility checks and a registered canary;
7. resume only if all predeclared gates pass.

Proposition 13.1 (Recovery locality)**.** *Under carrier-local restoration and a dependency-complete cache graph, a policy-only failure can be recovered without changing authoritative memory content, and a memory-only failure can be recovered without changing policy content. Cache eviction and pin refusal may affect availability but not carrier identity.*

Counterexample 13.2 (Rollback illusion from an incomplete snapshot)**.** A policy rollback restores adapter weights but not optimizer moments. The next update diverges. Terminal weights initially match the prior checkpoint, yet the restored state is not the prior policy carrier. A weight-file hash alone is therefore not a rollback receipt.

Counterexample 13.3 (Memory fieldwise undo)**.** A system restores a row's semantic payload but leaves the newer version, ACL, and provenance. The row is neither the old complete version nor a valid new one. This violates complete-row rollback even if a query happens to return the older fact.

13.4 Severe-event stop rules

Average task loss cannot override a severe integrity event. The protocol stops the affected stage on any cross-tenant read/write, unverified deletion, direct privilege crossing, future-information leak, silent cross-carrier restoration, lost mandatory receipt, or corruption of the frozen evaluation boundary. A thresholded task or cost gate may also stop for futility. These rules are design prescriptions, not evidence that a future implementation is safe.

14 Staged Falsification Protocol and Obligations

14.1 The prerequisite gate is first

The present research chain stops before a dual-state experiment. The same-checkpoint semantic interface is failed/unqualified, and predictive admission is closed. A valid future program must reopen no downstream stage until its prerequisite has independently qualified.

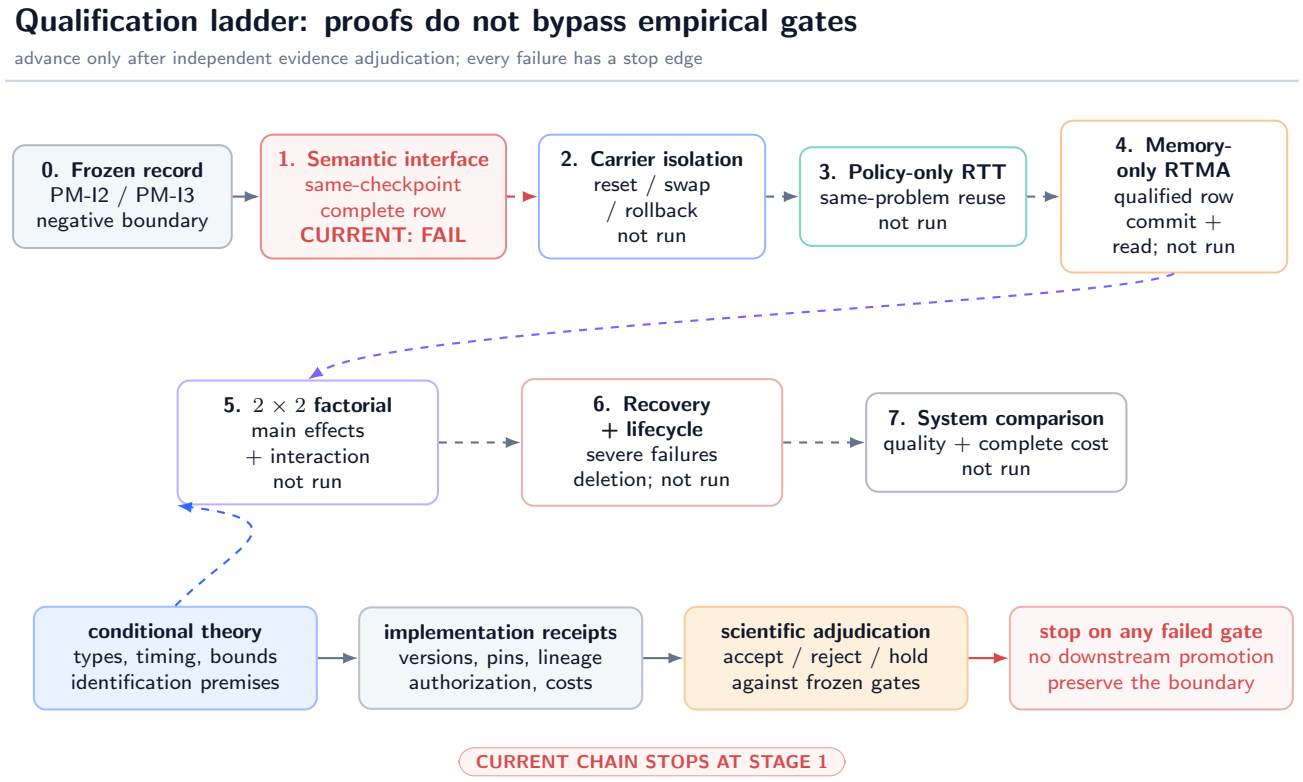

Figure 10: **Qualification ladder, stop edges, and proof dependencies (PLANNED TEST).** The coral semantic-interface stop is current. Theory boxes state conditional obligations; they do not bypass an empirical gate.

Protocol 14.1 (Dual-state qualification ladder)**.** Every stage uses immutable registration, fresh groups, required seeds, independent receipts, and explicit scientific adjudication.

1. **Semantic carrier qualification.** Establish a same-checkpoint complete typed row interface under its registered fit, held-out composition, query, preservation, mapping, and integrity gates. Current status: FAIL / unqualified.
2. **Carrier isolation.** Demonstrate independent snapshot, fresh/reset, swap, freeze, rollback, cache invalidation, and ledger behavior for Φ and M. Stop on any silent cross-carrier mutation.
3. **Policy-only strict RTT.** With M frozen, show feedback update, same-unresolved-problem later reuse, and a randomized policy intervention under fixed attempt/resource budgets. This is not a memory result.
4. **Memory-only RTMA.** With Φ frozen, show complete-row commit/NULL, later reuse, and negative controls. This is not a policy-learning result and requires a qualified memory interface.

5. 2×2 **factorial.** Randomize both carrier interventions, report all cell means, main effects, interaction, compliance, order, subgroup, severe-event, and cost results.
6. **Recovery and lifecycle.** Exercise carrier-local rollbacks, expiration, deletion, repeated updates/commits, and induced failure cases. Stop on unresolved contamination or receipt gaps.
7. **System comparison.** Only after all prior gates, compare against registered context, recurrent/compress-all, policy-only, memory-only, and static baselines on a Pareto ledger.

14.2 Primary endpoints and analysis

The primary scientific endpoint must be a task loss that does not reward direct answer caching. Co-primary integrity endpoints include semantic row exactness, preservation, policy drift, severe failures, and compliance. Resource endpoints retain the vector in Equation (26). The unit of randomization is the independent problem group; all attempts and branches from a group remain in one split. Final analysis reports uncertainty and multiplicity for the registered primary contrasts. Appendix B gives an executable design specification without inventing sample sizes or effect magnitudes.

14.3 Theorem-to-receipt obligations

Result	Minimum implementation receipt	Failure consequence
Type non-conflation	carrier tags, distinct credentials/endpoints, authorization log	type-safety proposition unusable
Causal timeline	event order, information-set schema, exact pinned predecessor view	temporal claim invalid
Conditional commutation	invariant inputs/decisions under both orders, compatibility receipts	order must be modeled as treatment
Strict RTT classification	unresolved marker, feedback, policy successor, later pinned descendant, intervention	call it retry/adaptation, not strict RTT
Independent rollback	complete carrier snapshot, lineage, cache dependency graph, unchanged other-carrier hash	recovery claim invalid
Contamination bound	step sizes, gradient bounds, Lipschitz region, fixed contaminant definition	bound cannot be cited
Factorial identification	randomized assignments, positive support, pins, compliance, group isolation, attrition log	contrasts descriptive only
Cost identity	event-complete units, failed work, cache proof, measurement method	efficiency claim prohibited

Table 9: Proofs become system statements only with their premises and receipts.

14.4 Decision grammar

Each stage ends in one of four durable scientific statuses: PASS under every registered gate; FAIL under at least one decisive gate; HOLD for incomplete but recoverable evidence; or INVALID for protocol corruption. Editorial completion and a clean PDF build have no bearing on those scientific statuses.

PLANNED TEST No stage of Protocol 14.1 was executed for this paper. In particular, no dual-state checkpoint, policy update, memory commit, factorial unit, oracle gap, or learned admission decision exists in the evidence record.

15 Related Work, Limitations, and Conclusion

15.1 Test-time updates and numerical state

Test-time training updates model parameters on test inputs under a self-supervised objective [9]. TTT layers treat an internal model as recurrent hidden state updated by learning at test time [8]. LoRA provides a parameter-efficient adapter representation for numerical adaptation [3]; it does not itself define when an update is reasoning-time, which feedback is valid, or how rollback should interact with authoritative memory. Recent LLM work performs test-time reinforcement learning from model-derived feedback [15]; Policy of Thoughts and StarOR describe transient within-instance policy adaptation for reasoning or optimization [4, 5]. These papers motivate explicit policy-state timing. Their empirical findings are not transferred to MMLA-RTT.

15.2 Recurrent and neural memory

Compressive Transformers compress older activations for long-range sequence modeling [6]; Mamba uses selective state-space recurrence [2]; TTT layers make the recurrent state a learned model [8]; and Titans proposes neural long-term memory updated at test time [1]. These are numerical or architectural memory mechanisms under their own contracts. Our use of "authoritative memory" is narrower: a bounded map of complete typed, versioned, provenance-bearing rows with trusted assembly and exact commit/NULL. The distinction is semantic and operational, not a claim of superior accuracy or efficiency.

15.3 Causal factorial attribution

The potential-outcome framework separates assignment from outcomes and makes interaction a contrast over unobserved potential responses [7]. Our contribution is to specialize that discipline to independently pinned policy and authoritative-memory carriers, with explicit noncompliance, swap, reset, rollback, sampler, cache, and ledger controls. Randomization can identify registered contrasts; it cannot validate a leaked endpoint, an unqualified semantic interface, or a missing receipt.

15.4 Limitations

This paper has deliberately strong limitations.

1. It is a theory/protocol paper and contains no new measurement.
2. The authoritative semantic interface required by the composition is currently failed/unqualified under the frozen evidence record.
3. The formal results rely on measurability, authorization, Lipschitz, capacity, compatibility, randomization, compliance, and ledger premises that no implementation has established.
4. The output-trace theorem allows alternate carriers with sufficient finite capacity; it is an identification result, not a realistic cost equivalence.
5. The contamination bound is worst-case and unsigned. It neither estimates real contamination nor proves that gating helps.
6. The 2×2 design does not solve endpoint validity, interference outside the chosen group, noncompliance, low power, or multiplicity by itself.
7. Independent operation algebras reduce semantic coupling but can increase coordination, storage, validation, and recovery cost.
8. We do not claim priority over all dual-state, adaptive-memory, recurrent, or test-time-learning systems.

15.5 Conclusion

The central design rule is simple but demanding: policy state Φ and authoritative memory M may cooperate, yet they must never become indistinguishable. They require separate types, writers, readers, scopes, versions, lineages, resets, swaps, rollbacks, ledgers, costs, and causal interventions. A pinned read view coordinates two independently owned states; it does not merge them. Strict RTT requires feedback-driven policy update and later same-unresolved-problem reuse. Memory mutation alone is RTMA. Output improvement alone certifies neither.

The resulting theory offers conditional guarantees and explicit failure tests, not a system result. Current evidence still stops at the failed/unqualified semantic interface. Any future dual-state claim must first clear that gate, then survive carrier-isolation controls, policy-only and memory-only qualification, the full factorial, recovery and lifecycle tests, and complete cost accounting. Until then, MMLA-RTT remains a falsifiable architecture proposal rather than an empirical success.

References

[1] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.

[2] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *International Conference on Learning Representations*, 2024.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[4] Zhengbo Jiao, Hongyu Xian, Qinglong Wang, Yunpu Ma, Zhebo Wang, Zifan Zhang, Dezhang Kong, and Meng Han. Policy of thoughts: Scaling LLM reasoning via test-time policy evolution. *arXiv preprint arXiv:2601.20379v2*, 2026.

[5] Jiajun Li, Yu Ding, Shisi Guan, Ran Hou, and Wanyuan Wang. StarOR: Synergizing tree search and test-time reinforcement learning for optimization modeling. *arXiv preprint arXiv:2606.15197v2*, 2026.

[6] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. *International Conference on Learning Representations*, 2020.

[7] Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 66(5):688–701, 1974.

[8] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

[9] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248, 2020.

[10] Junyi Zou and Avrova Donz. Atomic memory rows: A bounded, verifiable substrate for editable reasoning. MMLA Focus Paper; cited for the formal row contract, 2026.

[11] Junyi Zou and Avrova Donz. Learning what to remember: Predictive admission for bounded reasoning-time memory. MMLA Focus Paper; cited for the formal admission problem, 2026.

[12] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future. *arXiv preprint arXiv:2606.28876v3*, 2026.

[13] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future, complete technical report. Technical report, MMLA-org, 2026. Complete technical-report evidence record.

[14] Junyi Zou and Avrova Donz. Reasoning-time training: Learning before a single problem ends. MMLA Focus Paper; cited for formal terminology, 2026.

[15] Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, Biqing Qi, Youbang Sun, Zhiyuan Ma, Lifan Yuan, Ning Ding, and Bowen Zhou. TTRL: Test-time reinforcement learning. In *Advances in Neural Information Processing Systems 38*, 2025.

A Proofs and Constructions

A.1 Type and timing results

Proof of Proposition 2.5. Each mutation event contains a carrier tag. By Assumption 2.4, authorization for a Φ-tagged event is defined only on the policy carrier type and its successor map has codomain $\mathcal{P} \times \mathcal{O}$; authorization for an M-tagged event is defined only on the authoritative-memory type and has codomain $\mathcal{M}$. Induct on trace length. The empty trace is well typed. If the first n events leave both carrier projections well typed, event $n+1$ either changes only the carrier named by its tag or is rejected. Thus it cannot apply its payload to the other carrier. Serialized-byte equality does not change the tag, domain, or codomain. The property holds for every finite trace. □

Proof of Proposition 3.2. Proceed by induction on event order. The initial registered state and view are $\mathcal{F}_0$-measurable. Suppose S_e and all prior receipts are $\mathcal{F}_e$-measurable. Assumption 3.1 makes κ_e, its authorized input ξ_e, and any registered randomness measurable with respect to $\mathcal{F}_e$ or a declared independent random seed revealed at e. Because T_{κ_e} is a measurable typed map, S_{e+1} and its receipt are measurable with respect to $\mathcal{F}_{e+1}$. Induction gives the result. A version created after e is not in $\mathcal{F}_e$, so it cannot be a measurable parent of the output at e under the declared structural trace. □

A.2 Operation algebra

Proof of Theorem 4.3. Write the carrier projection of S as (P, A). By input stability, U receives the same policy input and authorization decision whether evaluated before or after K; hence its policy successor is one fixed $P^+ = U(P)$. Symmetrically, the memory candidate, validation, and authorization are invariant, so its successor is one fixed $A^+ = K(A)$. Typed authorization makes $\widetilde{U}$ leave A unchanged and $\widetilde{K}$ leave P unchanged. Consequently both orders have carrier projection (P^+, A^+). Receipt events preserve their temporal order; normalization may yield an equivalent set of receipts without identical serialized traces, so no full-trace equality is asserted. □

Proof of Proposition 4.5. Fix visible version numbers (a, b). Construct policy histories H_1^Φ and H_2^Φ with distinct lineage identifiers: in the first, update directly from baseline to version a; in the second, fork at an earlier snapshot, perform a reset, and publish content-compatible version a. Pair each with the same memory history ending at (b, ℓ^M). The pinned version pair is identical but the policy predecessor and rollback target differ. The symmetric construction holds for two memory histories paired with one policy history. A passing compatibility predicate depends only on its declared fields and therefore need not reveal either hidden predecessor. Thus the pair and receipt do not determine lineages or rollback targets. □

A.3 Mechanism classification

Proof of Theorem 5.5. Define binary indicators I_Φ and I_M for the strict policy path and later read of updated authoritative memory, respectively. Under observability, both are functions of the audit trace and are not imputed from output quality. The Cartesian set $\{0,1\}^2$ partitions traces into four disjoint cells. Definitions 5.1–5.3 name these cells static/retry, RTMA only, strict RTT only, and dual-state composition. Exhaustiveness follows because each indicator is binary; exclusivity follows because two distinct cells disagree in at least one indicator. The definitions do not include an outcome sign, so no benefit follows. □

Proof of Corollary 5.6. If Φ is frozen, no authorized feedback-driven policy successor exists and $I_\Phi = 0$. If a later attempt reads updated M, then $I_M = 1$. Theorem 5.5 assigns the interval to RTMA only. If there is no later memory read either, it is static/retry. Neither case is strict RTT. □

A.4 Restoration and lineage

Proof of Theorem 6.2. Assumption 6.1 defines Rollback_Φ as a map on the policy carrier and its ledger, with the identity map on the memory carrier. Typed authorization prevents its event from naming M as a destination. Hence $\pi_A(\text{Rollback}_\Phi(S)) = \pi_A(S)$, including content, version, and lineage. The memory case is symmetric. Compatibility evaluation is a later coordinator map whose outcomes are pin or refusal; it has no carrier-write privilege. Therefore a compatibility failure cannot alter the unchanged carrier. □

Proof of Corollary 6.3. By Theorem 6.2, one carrier-local rollback changes at most one carrier. Changing both therefore requires the composition of two authorized carrier-local maps. An explicit composite manifest is conforming only if its execution expands to those maps and retains both receipts. An unnamed single event cannot satisfy the typed domain requirement. □

Proof of Proposition 6.4. A restore produces a new lineage edge labeled reset, swap, or rollback whose successor has a new version and receipt. Append-only storage prevents deletion of that edge or the intervening vertices. Content addressing may show that the restored payload equals an earlier payload, but the new vertex and edge remain. An auditor reading the ledger therefore distinguishes restoration from uninterrupted continuation. □

A.5 Compression and drift

Proof of Theorem 7.2. Let $d_t = \|\Phi_t - \Phi_t^I\|$. Nonexpansiveness of projection and the triangle inequality give, for informative t,

$$d_{t+1} \le d_t + \eta_t \|g(\Phi_t, X_t) - g(\Phi_t^I, X_t)\| \le (1 + \eta_t L_g) d_t.$$

For contaminant t, the coupled informative path takes a zero update, so

$$d_{t+1} \le d_t + \eta_t \|g(\Phi_t, X_t)\| \le (1 + \eta_t L_g) d_t + \eta_t B_t,$$

where the last step adds and subtracts $g(\Phi_t^I, X_t)$. With $d_0 = 0$, unrolling this recurrence yields Equation (15). Lipschitz continuity of ℓ gives Equation (16). Absolute value removes the sign, so harm does not follow. □

Proof of Proposition 7.5. Counterexample 7.3 supplies a law and nonnegative cost under which compress-all sufficient-statistic recurrence is Bayes-optimal and an added carrier cannot strictly improve total risk. Conversely, choose a finite-capacity compress-all state whose update overwrites the only bit needed for a later task when a nuisance event arrives; a typed protected row retains that bit and achieves lower task loss at sufficiently small row cost. Since each architecture is strictly better in one admissible construction, no uniform dominance holds without additional restrictions. □

Proof of Proposition 8.1. Because $\Phi_k \in \mathcal{P}$ and projection fixes points in $\mathcal{P}$,

$$\|\Phi_{k+1} - \Phi_k\| = \|\mathrm{proj}_{\mathcal{P}}(\Phi_k - \eta_k \widehat{g}_k) - \mathrm{proj}_{\mathcal{P}}(\Phi_k)\| \le \eta_k \|\widehat{g}_k\| \le \eta_k G_k.$$

Sum over k and apply the triangle inequality. The stated total-variation bound follows directly from its Lipschitz premise. □

Proof of Theorem 8.3. Substitute the intervention bounds $\delta_\Phi \le \epsilon_\Phi$ and $\delta_M \le \epsilon_M$ into Equation (19); all coefficients and distances are nonnegative. For integrable bounded losses, take expectations and use $|\mathbb{E}X| \le \mathbb{E}|X| \le B_{\Phi M}$. The inequality is absolute and supplies no sign or additive causal attribution. □

A.6 Factorial and observational results

Proof of Theorem 9.3. Random assignment and positive support imply $Z_i \perp \{Y_i(p,m) : p, m \in \{0,1\}\}$. Perfect compliance and consistency give $Y_i^{\mathrm{obs}} = Y_i(p,m)$ in cell (p,m). Hence

$$\mathbb{E}[Y_i^{\mathrm{obs}} \mid Z_i^\Phi = p, Z_i^M = m] = \mathbb{E}[Y_i(p,m)] = \mu_{pm}.$$

Linear combinations of the four identified cell means identify the displayed main and interaction effects. For known probabilities $\pi_{pm} > 0$, the Horvitz–Thompson term $\mathbf{1}\{Z = (p,m)\}Y^{\mathrm{obs}}/\pi_{pm}$ has expectation μ_{pm}, proving the final statement. □

Proof of Proposition 9.4. Let D be successful exposure. Even when Z is randomized, D can depend on assignment, latent problem difficulty, resource pressure, or potential outcomes. Conditioning on $D = 1$ opens selection paths and generally destroys $Z \perp Y(p,m)$. Thus receipt-positive differences are not automatically randomized effects. Guaranteed compliance or an explicit identification model is required. □

Proof of Theorem 10.1. Because the observable alphabet and horizon are finite, a policy-update system induces a conditional kernel $K(z_1 \mid q, z_0, f_0)$. Construct a memory-only system whose frozen policy reads a row containing a finite table encoding this kernel. After (q, z_0, f_0), the memory updater writes the corresponding complete table row; the later frozen policy samples from it using the same exogenous randomness. The induced observable law equals the original. For the converse, store the finite memory-update table in a sufficiently large numerical policy state and freeze authoritative memory. A policy update writes the table entry, and the later policy emits according to the same kernel. These constructions preserve the observable law but not cost or semantics. Therefore the output trace cannot identify the changed carrier. □

Proof of Corollary 10.2. For one unit, consistency reveals only $Y_i(Z_i^\Phi, Z_i^M)$. The interaction contains all four potential outcomes. Without structural restrictions that determine the three unobserved values, multiple potential-outcome schedules agree on the observed trajectory and yield different interactions. Hence one trajectory does not identify it. □

Proof of Proposition 10.6. Under randomized valid swap assignment, donor content is independent of recipient potential outcomes and all declared non-content fields are fixed or balanced. Consistency therefore identifies the mean outcome under recipient-updated content and under a draw from the declared compatible-donor distribution. Their difference is the recipient-specific-content contrast relative to that distribution. Nothing in the assumptions ranges over every possible donor or semantic representation, so no universal effect follows. □

A.7 Authorization, cost, and recovery

Proof of Proposition 11.2. Distinct credentials and typed endpoints make the policy service unauthorized at the authoritative commit endpoint and make the candidate/committer service unauthorized at the policy endpoint. Schema checks reject a payload carrying the wrong carrier type. Therefore a call confined to one ordinary service cannot directly publish the other's successor. The conclusion does not survive compromise of the shared authorization root or both credentials, which the proposition excludes. □

Proof of Proposition 12.1. The five event sets form a disjoint partition of all charged operations. Finite additivity of vector sums gives Equation (27). Each listed activity is, by construction, assigned to one set, including failures and recovery. Reporting only generation projects away non-generation components and therefore cannot equal the complete vector unless every omitted component is proved zero. □

Proof of Corollary 12.2. Parallel scheduling changes the temporal arrangement of event costs and can reduce the wall-time coordinate. It does not remove executed events from the partition or make their logical counts and transferred bytes vanish. The corresponding vector coordinates remain summed. □

Proof of Proposition 13.1. By Theorem 6.2, carrier-local restoration changes only the named carrier. A dependency-complete cache graph identifies every derived cache entry and permits invalidation without altering source carrier content. Compatibility checks can withhold availability. Thus recovery of one carrier need not change the other's content, though temporary service interruption may occur. □

A.8 Proof boundary

Every result above is conditional on its displayed premises. None establishes a qualified semantic interface, actual privilege separation, valid feedback, bounded gradients, Lipschitz behavior, randomized compliance, a measured endpoint, or a complete ledger. Those are empirical or implementation obligations in Appendix C.

B Registered Factorial Protocol Specification

B.1 Purpose and gate order

This appendix is an unexecuted protocol template. It cannot begin until the semantic-interface gate and carrier-isolation stage in Protocol 14.1 pass. It estimates policy and memory intervention effects without treating assignments, successful pins, or final outputs as interchangeable.

B.2 Immutable registration

Before any evaluation outcome is opened, register:

1. problem-group construction, inclusion/exclusion rules, immutable group identifiers, and train/development/final split hashes;
2. base model, tokenizer, precision, software/hardware identity, causal workspace schema, maximum context and attempt budget;
3. exact Φ_0, Φ_1, M_0, M_1 construction, snapshot hashes, lineages, lifetimes, and compatibility rules;
4. update objectives, feedback sources, optimizer, step schedule, clipping/projection, memory candidate/validator/commit contract, and event order;
5. primary task loss, semantic and preservation gates, severe-event endpoints, subgroups, cost vector, materiality/equivalence margins, and missing-data rules;
6. randomization algorithm, assignment probabilities, blocking variables, seed list, multiplicity family, interval method, and stop rules;
7. exact analysis code or a content-addressed specification sufficient for independent reconstruction.

No sample-size number or effect magnitude is supplied here because none is supported by the frozen record. A future design must derive group count from a preregistered smallest effect of interest, variance pilot that is isolated from final evaluation, desired power, familywise error, expected noncompliance, and severe-event precision.

B.3 Unit and treatment construction

The randomization unit is a problem group containing every attempt, branch, paraphrase, and derived query that shares latent content or an upstream source. Units are blocked on preregistered difficulty and domain variables, then assigned to four cells

with positive probability. Assignment occurs before the evaluated attempt and is recorded independently from execution services.

Cell	Policy treatment	Memory treatment	Pinned-view requirement
00	registered fresh/frozen baseline Φ_0	registered fresh/frozen baseline M_0	exact (v_0^Φ, v_0^M)
10	feedback-updated Φ_1	baseline M_0	exact (v_1^Φ, v_0^M)
01	baseline Φ_0	qualified committed M_1	exact (v_0^Φ, v_1^M)
11	feedback-updated Φ_1	qualified committed M_1	exact (v_1^Φ, v_1^M)

Table 10: Primary availability factorial. M_1 does not exist as qualified evidence in this paper.

Exact NULL is an outcome of the registered memory transition, not missing data. If a cell assigned to memory update receives NULL, assignment remains $Z^M = 1$ while realized exposure is recorded separately. Failed pins, timeouts, validator rejects, and recovery events remain in intent-to-treat analysis under registered loss/missingness rules.

B.4 Execution sequence

For each unit:

1. verify immutable registration and cold/fixed evaluation boundary;
2. materialize carrier baselines from registered snapshots;
3. run the common pre-intervention attempt and record admissible feedback;
4. execute independently authorized policy and memory events in the assigned order;
5. verify both successor receipts, caches, compatibility, and exact pinned view;
6. run the evaluated attempt under the fixed sampler and resource contract;
7. compute endpoints with an evaluator blind to cell labels where feasible;
8. append all physical and logical costs, including failures and recovery;
9. freeze final group bytes before opening aggregate cell summaries.

If the policy and memory events may not commute, randomize their order within the joint arm or treat order as a registered additional factor. Do not average undeclared interleavings.

B.5 Primary estimators

For equal-probability complete randomization, let $\widehat{\mu}_{pm}$ be the group mean in cell (p, m). Estimate

$$\widehat{\tau}_{\Phi|m} = \widehat{\mu}_{1m} - \widehat{\mu}_{0m}, \tag{28}$$

$$\widehat{\tau}_{M|p} = \widehat{\mu}_{p1} - \widehat{\mu}_{p0}, \tag{29}$$

$$\widehat{\tau}_{\Phi M} = \widehat{\mu}_{11} - \widehat{\mu}_{10} - \widehat{\mu}_{01} + \widehat{\mu}_{00}. \tag{30}$$

Use randomization-based or group-robust uncertainty consistent with the assignment design. With unequal probabilities, use registered inverse-probability weights. Covariate adjustment may improve precision only if specified before final outcomes and if unadjusted estimates remain reported. The primary estimand is intent-to-treat; exposure/compliance analyses are secondary unless compliance is guaranteed.

B.6 Negative controls

The protocol includes:

1. same carrier versions with independent sampler seeds;
2. frozen policy with identical extra attempt and feedback-computation budget;
3. swapped policy with matched update count, base model, tokenizer, optimizer schema, and budget;
4. swapped memory with matched schema, tenant-safe synthetic scope, row count, version depth, and bytes;
5. independent policy rollback with unchanged memory and independent memory rollback with unchanged policy;
6. invalid/mismatched compatibility canaries that must block pinning;
7. target-revealing feedback canaries that must fail the leakage gate;
8. exact NULL and no-op controls whose authoritative resident bytes remain identical.

B.7 Stop and decision rules

Stop immediately for cross-tenant access, future-information leakage, direct cross-carrier write, silent joint reset/rollback, unverified deletion, receipt corruption, or evaluation-boundary mutation. Stop the scientific stage if the semantic, preservation, strict-RTT timing, intervention compliance, severe-event, or resource gate fails. A favorable task mean cannot override these stops. A passing factorial requires all registered gates; a non-significant interaction is not automatically equivalence unless its confidence set lies within the registered equivalence margin.

B.8 Reporting

Report all four cell summaries; conditional and marginal main effects; interaction; assignment and exposure counts; NULL, failure, timeout, and rollback counts; subgroups; severe events; update/commit order; drift; every cost coordinate; exact software/hardware and carrier hashes; and all deviations. The report must state that the design does not establish a learned admission policy unless a separately qualified admission stage exists.

C Theorem-to-Implementation Obligation Matrix

Table 11: Every conditional result remains unusable as a system claim until its complete premise bundle is evidenced.

Claim/result	Formal premise	Required durable evidence	If absent
Policy carrier identity	typed $\mathcal{P} \times \mathcal{O}$, version and lineage	complete parameter/optimizer manifest, scope, writer credential, predecessor/successor hashes	policy update/reset/rollback claim invalid
Memory carrier identity	complete typed rows and protected fields	serializer/assembler version, full resident snapshot, row/slot versions, ACL/provenance/retention receipts	authoritative-memory claim invalid
Pinned read view	exact versions remain stable during attempt	pin receipt, read-service logs, cache version, compatibility decision	attempt exposure unknown
Type safety	carrier-specific domains and privileges	distinct endpoints/credentials, negative authorization tests, carrier-tag schema	syntactic proposition not implemented
Causal measurability	inputs available at event time	timestamp/order source, information-set schema, future-access guards, sampler seed	leakage not excluded
Conditional commutation	input and decision stability in both orders	replay of both orders from identical snapshot, candidate/gradient equality, compatibility receipts	order is a treatment
Strict RTT	feedback update and later same-problem read	unresolved marker, feedback receipt, policy update lineage, later pin, frozen/reset intervention	call only retry or unclassified adaptation
RTMA	memory mutation and later read	complete commit/NULL receipt, memory lineage, later pin, frozen/reset intervention	memory mechanism unclassified
Independent rollback	carrier-local restore and cache graph	complete snapshot, new restore successor, unchanged other-carrier hash, cache invalidation	recovery claim invalid
Append-only history	durable content-addressed lineage	storage/ledger integrity proof, restore edge, retained intervening vertices	rollback can be observationally erased
Contamination bound	nonexpansive projection, Lipschitz gradients/loss, bounded contaminant updates	registered region, measured/bounded constants, fixed contaminant definition, update trace	theorem cannot bound deployment
Policy path length	bounded gradients and steps	exact step sizes, gradient norms, projection implementation, checkpoints	displacement bound unavailable
Interaction envelope	registered state metrics and constants	intervention radii, mixed-sensitivity validation on region	envelope unavailable
Factorial identification	randomization, support, consistency, no interference, compliance	assignment receipt, group construction, pins, exposure, attrition, frozen analysis	cell contrasts descriptive
Swap specificity	valid donor sampling and compatibility	donor-selection seed, no-leak proof, matched metadata, independent snapshot	content contrast confounded

Continued on next page

Claim/result	Formal premise	Required durable evidence	If absent
Output non-identification	finite output interface and alternate capacity	mathematical construction only	prohibits mechanism inference from output alone
Privilege isolation	distinct credentials and trusted mediation	authorization topology, penetration/negative tests, candidate quarantine	contamination-blocking claim unavailable
Cost identity	event-complete partition and units	per-event vector, failed work, cache proof, measurement calibration	efficiency claim prohibited
Recovery locality	independent restore plus complete cache graph	dependency inventory, carrier hashes before/after, availability log	other carrier may have changed
Semantic qualification	registered same-checkpoint interface gates	independent fit and held-out, query, preservation, mapping, and integrity adjudication	every memory-composition stage remains closed
Learned admission	qualified interface, oracle gap, future-blind value and selector gates	separate predictive-admission protocol receipts and accepted evidence	no learned policy claim

The final two rows are scientific prerequisites rather than premises of the purely mathematical carrier algebra. They prevent formal consistency from being misreported as an implemented or learned MMLA system.

D Symbols, Status, and Evidence Ledger

D.1 Core symbols

Symbol		Meaning
q, r, t, e		problem, attempt, within-attempt micro-event, and global event indices
$\mathcal{F}_e$		information available immediately before event e
$W_{q,r,t}$		bounded causal workspace; excludes future information
P $(\Phi, O, v^\Phi, \ell^\Phi, \tau^\Phi, \rho^\Phi)$	=	policy carrier: parameters, optimizer, version, lineage, lifetime, receipt
A $(M, v^M, \ell^M, \tau^M, \rho^M)$	=	authoritative memory carrier and its metadata
$V_{q,r}$		pinned read-view tuple naming both carrier versions/lineages and compatibility receipt
C, G, Q		immutable causal context; generation/sampler state; queues, caches, snapshots, and ledgers
$\mathfrak{O}_\Phi, \mathfrak{O}_M$		independent carrier operation algebras
U, K		one policy update and one memory transition
$Q_q(r)$		indicator that problem q remains unresolved after attempt r
$Y_i(p, m), \mu_{pm}$		potential loss and population cell mean under carrier interventions (p, m)
$\tau_{\Phi\|m}, \tau_{M\|p}$		conditional policy and memory effects
$\tau_{\Phi M}$		factorial interaction $\mu_{11} - \mu_{10} - \mu_{01} + \mu_{00}$
Z_i^Φ, Z_i^M		randomized assignments
D_i^Φ, D_i^M		receipt-verified realized exposures/compliance
$\mathbf{C}$		vector resource ledger
NULL		exact authoritative memory identity transition; rejection is externally receipted

D.2 Status ledger

Object	Status in this paper	Boundary
Typed dual-state semantics	DEFINITION	editorial/mathematical construction
Type, timing, rollback, drift, contamination, interaction, and identification results	PROVED UNDER STATED ASSUMPTIONS	conditional on displayed assumptions
Semantic row interface	UNVALIDATED	inherited failed/unqualified same-checkpoint gate
Strict-RTT policy implementation	UNVALIDATED	no policy update or same-problem intervention
Authoritative memory composition	UNVALIDATED	no qualified row interface or dual-state commit
2×2 experiment	PLANNED TEST	no unit assigned or evaluated
Oracle gap and predictive admission	UNVALIDATED	not measured; gate closed
Learned admission policy	UNVALIDATED	no value model, selector, or policy
Dual-state benefit/safety/efficiency	UNVALIDATED	no empirical result

Table 13: No status is promoted by manuscript completion.

D.3 Frozen numerical evidence

Only the inherited negative/diagnostic boundary is numerically admitted here:

- PM-I2: $0/9$ registered interfaces qualified.
- PM-I3 direct candidate: $40/73{,}728$ exact.
- Direct present path: 0.
- Learned route/learned content: $0/73{,}728$.
- Oracle route/learned content: $0/73{,}728$.
- Learned route/oracle content: $18{,}544/73{,}728$, exactly routing count.
- Oracle route/oracle content: $73{,}728/73{,}728$, mechanical ceiling only.
- Preservation: 0; missing cases: 43,008.

These counts delimit what is not qualified. They do not estimate any dual-state factorial cell, interaction, policy drift, contamination rate, resource cost, oracle gap, or admission effect.

D.4 Editorial boundary

Typeset completeness, a proof appendix, and visual quality establish only editorial integrity. They do not qualify a mechanism or promote a scientific claim.